UFMT

UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

AVALIAÇÃO EXPERIMENTAL E COMPARATIVA DE MEDIDAS PARA ATENUAR ATAQUES DE FORÇA BRUTA EM SERVIDORES SSH

WELLINGHTON FERNANDO ARMOA PIMENTA

CUIABÁ - MT

2026

UFMT

UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

AVALIAÇÃO EXPERIMENTAL E COMPARATIVA DE MEDIDAS PARA ATENUAR ATAQUES DE FORÇA BRUTA EM SERVIDORES SSH

WELLINGHTON FERNANDO ARMOA PIMENTA

Orientador: Prof. Dr. Nielsen Simões

Monografia apresentada ao Curso de Ciência da Computação, do Instituto de Computação da Universidade Federal de Mato Grosso, como requisito para obtenção do título de Bacharel em Ciência da Computação

CUIABÁ - MT

2026

Wellinghton Fernando Armoa Pimenta

Avaliação Experimental e Comparativa de Medidas para Atenuar Ataques de Força Bruta em Servidores SSH

Trabalho aprovado.

Cuiabá-MT, 24 de Setembro de 2025

Prof. Dr. Nielsen Simões
Orientador

Prof. Dr. Jean Caminha
Convidado 1

Prof. Dr. César Eduardo Guarienti
Convidado 2

Este trabalho é dedicado a todos que um dia sonharam em ser tornar cientistas.

AGRADECIMENTOS

A Deus ...

Aos professores do Instituto de Computação, em especial, aos professores Nielsen e Claudia, pela ajuda nas correções e pelas valorosas sugestões

A todos que de alguma forma contribuíram para o desenvolvimento deste modelo de monografia.

RESUMO

Este Trabalho de Conclusão de Curso apresenta uma investigação sobre técnicas, ferramentas e boas práticas destinadas a aumentar a robustez de servidores SSH contra tentativas automatizadas de acesso não autorizado. Inicialmente, realiza-se uma revisão bibliográfica que descreve o estado da arte em mitigação de ataques de força bruta, contemplando soluções como Fail2Ban, DenyHosts, SSHGuard, port knocking e autenticação por chave pública. Em seguida, apresenta-se a metodologia empregada, que inclui a configuração de um ambiente de testes em Linux, a implementação e parametrização das ferramentas de defesa, a simulação de ataques de força bruta com ferramentas como Hydra e Medusa, bem como a coleta e análise de logs para avaliação de desempenho e eficácia das soluções. Os resultados esperados visam identificar quais medidas oferecem melhor relação entre segurança, desempenho e facilidade de implementação. Por fim, discute-se a aplicabilidade prática das recomendações, destacando-se a necessidade de ajustes conforme o perfil de risco de cada infraestrutura.

Palavras-chaves: SSH; Força Bruta; Segurança de Servidores; Fail2Ban; Port Knocking; Hydra; Medusa

SUMÁRIO

1 INTRODUÇÃO . . . 1
1.1 Objetivo Geral . . . 2
1.2 Objetivos Específicos . . . 2
1.3 Justificativa . . . 2
1.4 Metodologia . . . 3

2 FUNDAMENTAÇÃO TEÓRICA . . . 4
2.1 Protocolo SSH e seus Mecanismos de Segurança . . . 4
2.1.1 Custos de Bytes de Rede em Tentativas de Conexão SSH . . . 5
2.2 Estrutura de Pacotes e Overhead de Protocolo . . . 6
2.2.1 Pacote SSH_MSG_USERAUTH_REQUEST (Cliente para o Servidor) . . . 6
2.2.2 Pacote SSH_MSG_USERAUTH_FAILURE (Servidor para o Cliente) . . . 6
2.2.3 Impacto em Cenários de Ataque . . . 7
2.3 Ataques de Força Bruta: Caracterização e Impactos . . . 8
2.3.1 Ferramentas Comuns para Ataques de Força Bruta . . . 8
2.3.1.1 Hydra . . . 9
2.3.1.2 Medusa . . . 10
2.4 A Evolução das Táticas de Ataque de Força Bruta . . . 11
2.4.1 Ataques de Baixa e Lenta Frequência (*Low-and-Slow*) . . . 11
2.4.2 Ataques de Pulverização de Senhas (*Password Spraying*) . . . 12
2.4.3 Ataques Baseados em Credenciais Vazadas (*Credential Stuffing*) . . . 12
2.5 Abordagens de Mitigação . . . 13
2.5.1 Controles Proativos . . . 14
2.5.1.1 Autenticação por Chave Pública . . . 14
2.5.1.2 Funcionamento e Vantagens Criptográficas . . . 14
2.5.2 Soluções Reativas . . . 15
2.5.2.1 Fail2Ban: O Padrão da Indústria . . . 15
2.5.2.2 Arquitetura e Mecanismo de Funcionamento . . . 15
2.5.3 Análise de Desempenho e Limitações . . . 16
2.5.4 SSHGuard . . . 16
2.5.4.1 Princípios de Funcionamento e Diferenças Chave . . . 17
2.5.5 Port Knocking . . . 17
2.5.5.1 Fundamentos Conceituais do Port Knocking . . . 17
2.5.5.2 Análise de Desafios e Soluções Avançadas . . . 18

2.5.6 Limitações Intrínsecas das Ferramentas Reativas 19
2.6 Estratégias de Defesa em Camadas e a Sinergia entre Ferramentas 20
2.6.1 O Princípio da Defesa em Camadas Aplicado ao SSH 21

3 METODOLOGIA 22
3.1 Arquitetura do Ambiente de Teste 22
3.1.1 Plataforma de Virtualização 23
3.1.2 Maquina Virtual 1: Servidor Alvo 23
3.1.3 Maquina Virtual 2: Plataforma de Ataque 24
3.2 Configuração dos Cenários Experimentais 24
3.2.1 Cenário de Controle Base 25
3.2.2 Cenários de Mitigação Reativa 25
3.2.3 Cenário de Mitigação por Ocultação (Port Knocking) 26
3.2.4 Cenário de Mitigação Proativa (Autenticação por Chave Pública) 26
3.2.5 Cenário de Defesa em Camadas (Synergistic Defense) 26
3.3 Protocolo de Simulação de Ataques 27
3.3.1 Dicionários de Ataque 27
3.3.2 Execução do Ataque de Alto Volume (Hydra) 27
3.3.2.1 Execução do Ataque de Alto Volume (Medusa) 28
3.4 Coleta e Análise de Dados 28
3.4.1 Definição das Métricas de Avaliação 29
3.4.1.1 Métricas de Eficácia de Segurança 29
3.4.1.2 Métricas de Impacto no Desempenho 29
3.4.2 Instrumentação e Procedimentos de Coleta 29
3.4.2.1 Coleta de Métricas de CPU e Memória com `htop` e `vmstat` 30

4 APRESENTAÇÃO DOS RESULTADOS 32
4.1 Apresentação dos Resultados 32
4.1.1 Análise de Eficácia das Medidas de Mitigação 33
4.1.2 Análise de Impacto no Desempenho do Servidor 36
4.1.3 Resumo dos resultados 39
4.2 Discussão dos Resultados 40
4.2.1 O Custo da Inação: Análise do Cenário Baseline 40
4.2.2 A Eficácia e o Overhead das Ferramentas Reativas 40
4.2.3 A Superioridade das Abordagens Proativas 41
4.2.4 Limitações do Estudo 41

5 CONCLUSÃO 42

REFERÊNCIAS 44

A CONFIGURAÇÃO DE UM JAIL PARA FAIL2BAN 47

B CONFIGURAÇÃO E INTEGRAÇÃO DO SSHGUARD 49

C CONFIGURAÇÃO E INTEGRAÇÃO DO PORT KNOCKING (KNOCKD) 51

D CONFIGURAÇÃO E INTEGRAÇÃO DE AUTENTICAÇÃO POR CHAVE PÚBLICA 54

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura em camadas do protocolo SSH. 5
Figura 2 – Ferramenta de Ataque de Força Bruta THC-Hydra 9
Figura 3 – Ferramenta de Ataque de Força Bruta Medusa 10
Figura 4 – Arquitetura de um Ataque de Força Bruta Distribuído via Botnet . . . 12
Figura 5 – Modelo de Defesa em Camadas para um Servidor SSH 20
Figura 6 – Configuração do Internal Network no VirtualBox 23
Figura 7 – Imagem do funcionamento da ferramenta htop 30
Figura 8 – Imagem do funcionamento da ferramenta vmstat 31
Figura 9 – Taxa de sucesso do ataque por cenário experimental. 33
Figura 10 – Número total de endereços IP bloqueados pelas ferramentas reativas. . 34
Figura 11 – Tempo para o bloqueio de um endereço IP atacante. 35
Figura 12 – Carga de CPU no servidor alvo durante o ataque. 36
Figura 13 – Consumo de memória RAM no servidor alvo durante o ataque. 37
Figura 14 – Volume total de tráfego de rede na porta SSH por cenário. 38
Figura 15 – Crescimento do arquivo de log de autenticação (`/var/log/auth.log`). 39

LISTA DE TABELAS

Tabela 1 – Estrutura do Pacote SSH_MSG_USERAUTH_REQUEST 6
Tabela 2 – Estrutura do Pacote SSH_MSG_USERAUTH_FAILURE 6
Tabela 3 – Padrões de Ataque de Força Bruta em SSH 8
Tabela 4 – Comparativo de eficácia e impacto no desempenho das ferramentas. . 39
Tabela 5 – Referência de Parâmetros do `knockd.conf` 53

LISTA DE ABREVIATURAS E SIGLAS

C&C	Comando e Controle
CPU	Unidade Central de Processamento
DoS	Negação de Serviço
FTP	Protocolo de Transferência de Arquivos
HTTP	Protocolo de Transferência de Hipertexto
IC	Instituto de Computação
IP	Protocolo de Internet
IPv4	Protocolo de Internet, Versão 4
IPv6	Protocolo de Internet, Versão 6
LTS	Suporte de Longo Prazo
NAT	Tradução de Endereços de Rede
PKA	Autenticação por Chave Pública
RAM	Memória de Acesso Aleatório
SPA	Autorização por Pacote Único
SSH	Protocolo de rede Secure Shell

TCP	Protocolo de Controle de Transmissão
THC	The Hacker's Choice
TOTP	Senha de Uso Único Baseada em Tempo
UFMT	Universidade Federal de Mato Grosso
VM	Máquina Virtual

CAPÍTULO 1

INTRODUÇÃO

O Secure Shell (SSH) é amplamente adotado como o protocolo padrão para acesso remoto seguro e transferência de arquivos em ambientes corporativos e na nuvem, devido à sua capacidade de oferecer autenticação forte e criptografia ponta a ponta (YLÖNEN; LONVICK, 2006). Contudo, essa popularidade também o torna um alvo atrativo para atacantes. Servidores SSH expostos à Internet enfrentam uma ameaça constante: ataques de força bruta automatizados, nos quais agentes maliciosos tentam incessantemente adivinhar combinações de usuário e senha até obter acesso não autorizado (ET Edge Insights, 2024).

Esse tipo de ataque representa um risco real e crescente à segurança de sistemas e redes. Estatísticas recentes revelam que, apenas no primeiro trimestre de 2024, honeypots ao redor do mundo registraram milhares de tentativas de login por força bruta, com um aumento de até 12% em relação ao ano anterior (AhnLab Security Intelligence Center (ASEC), 2025). Além de comprometer a integridade dos sistemas, esses ataques geram sobrecarga nos servidores, consomem recursos computacionais e podem abrir caminho para ameaças ainda mais graves, como movimentação lateral, exfiltração de dados e instalação de malwares persistentes.

Apesar da existência de diversas ferramentas e práticas de mitigação — como autenticação por chave pública, limitação de tentativas de login e uso de firewalls —, muitas

vezes essas medidas são implementadas de forma isolada e sem avaliação sistemática de sua efetividade diante de diferentes cenários de ataque.

Diante desse contexto alarmante, este trabalho propõe uma abordagem experimental que busca implementar, testar e comparar estratégias integradas de defesa contra ataques de força bruta em servidores SSH.

1.1 Objetivo Geral

Avaliar, por meio de experimentação controlada, a eficácia e o impacto em desempenho de diferentes medidas de segurança — incluindo Fail2Ban, SSHGuard, DenyHosts, port knocking e autenticação por chave pública — para atenuar ataques de força bruta em servidores SSH, identificando a combinação de técnicas que otimize a relação entre proteção e usabilidade.

1.2 Objetivos Específicos

1. Configurar um ambiente de testes virtualizado com Ubuntu Server e Kali linux para simulação de ataques de força bruta contra SSH.
2. Instalar e parametrizar as ferramentas Fail2Ban, SSHGuard e DenyHosts, ajustando número máximo de tentativas, janela de tempo e duração de bloqueio.
3. Implementar port knocking para ocultar a porta SSH até a apresentação de sequência pré-definida de “batidas” em portas.
4. Configurar autenticação por chave pública RSA, desabilitando o login por senha, e avaliar seu efeito isolado e combinado com as demais ferramentas.
5. Analisar quantitativamente os logs gerados, comparando eficácia de bloqueio e impacto no desempenho.

1.3 Justificativa

Ataques de força bruta a servidores SSH representam um dos principais vetores de invasão em ambientes críticos, podendo resultar em vazamentos de dados, instalação de malware e indisponibilidade de serviços essenciais (AhnLab Security Intelligence Center (ASEC), 2025). Embora haja diversas soluções pontuais, faltam estudos comparativos que quantifiquem como a combinação de múltiplas camadas de defesa afeta não apenas a segurança, mas também o desempenho e a complexidade operacional do servidor.

1.4 Metodologia

Esta seção tem por objetivo definir bases metodológicas sólidas sobre as quais se desenvolverá este Trabalho de Conclusão de Curso, descrevendo a natureza da pesquisa, métodos e técnicas que serão adotados. A fim de apresentar um maior nível de detalhamento, essa definição foi considerada quanto a quatro aspectos:

1. **Natureza da pesquisa**
   Trata-se de um trabalho de conclusão de curso de natureza *experimental aplicada*, pois envolverá a implementação prática e o teste de ferramentas e técnicas de segurança em servidores SSH, com o propósito de gerar conhecimento útil para aplicação em ambientes reais.

2. **Abordagem**
   Este estudo terá abordagem *quantitativa*, visto que coletará métricas numéricas precisas — como número de bloqueios, taxa de sucesso de login, latência e consumo de recursos — permitindo comparação objetiva entre diferentes configurações de segurança.

3. **Objetivos**
   Os objetivos do trabalho têm *natureza exploratória*, na medida em que visam identificar e comparar o impacto de diversas medidas de defesa (Fail2Ban, SSHGuard, DenyHosts, port knocking e autenticação por chave pública) sobre a resiliência do servidor SSH e seu desempenho operacional.

4. **Procedimentos Técnicos**

   - *Pesquisa bibliográfica*: revisão de artigos, livros e documentação técnica sobre mitigação de ataques de força bruta em SSH, incluindo estudos como (PARK et al., 2021).
   - *Pesquisa experimental*: configuração de ambiente virtualizado com Ubuntu Server e Kali Linux, instalação e parametrização de ferramentas de defesa, e execução de ataques de força bruta por meio de Hydra e Medusa.
   - *Coleta de dados*: registro automático de logs de autenticação, contagem de bloqueios, medição de latência de resposta e monitoramento de uso de CPU/-memória.
   - *Análise de dados*: aplicação de estatística descritiva e comparativa para avaliar compensações entre eficácia de bloqueio e impacto no desempenho, complementada por avaliação qualitativa da complexidade de implantação.

CAPÍTULO 2

FUNDAMENTAÇÃO TEÓRICA

Este capítulo consolida os referenciais teóricos que sustentam a pesquisa sobre medidas de segurança para atenuar ataques de força bruta em servidores SSH, organizando o conteúdo em três eixos principais: (2.1) a arquitetura e os mecanismos de segurança do protocolo SSH, (2.3) a caracterização dos ataques de força bruta, abordando suas técnicas, motivações e impactos em servidores expostos e (2.5) a apresentação e análise crítica de estratégias de mitigação voltadas especificamente para o endurecimento de servidores SSH.

2.1 Protocolo SSH e seus Mecanismos de Segurança

O *Secure Shell* (SSH), formalizado no (YLöNEN; LONVICK, 2006), é um protocolo de camada de aplicação que fornece comunicação criptografada em redes não confiáveis. Sua arquitetura divide-se em três componentes principais:

1. **Camada de Transporte**: Utiliza algoritmos como AES-256-GCM para criptografia simétrica e Diffie-Hellman para troca de chaves, garantindo confidencialidade e integridade.

2. **Camada de Autenticação**: Oferece dois métodos:

- **Senha**: Vulnerável a ataques de dicionário quando combinado com políticas fracas (e.g., senhas curtas ou comuns).
- **Chaves Públicas**: Padrão recomendado com eficácia dependente do uso de chaves RSA-4096 ou Ed25519.

3. **Camada de Conexão**: Multiplexação de canais para sessões remotas.

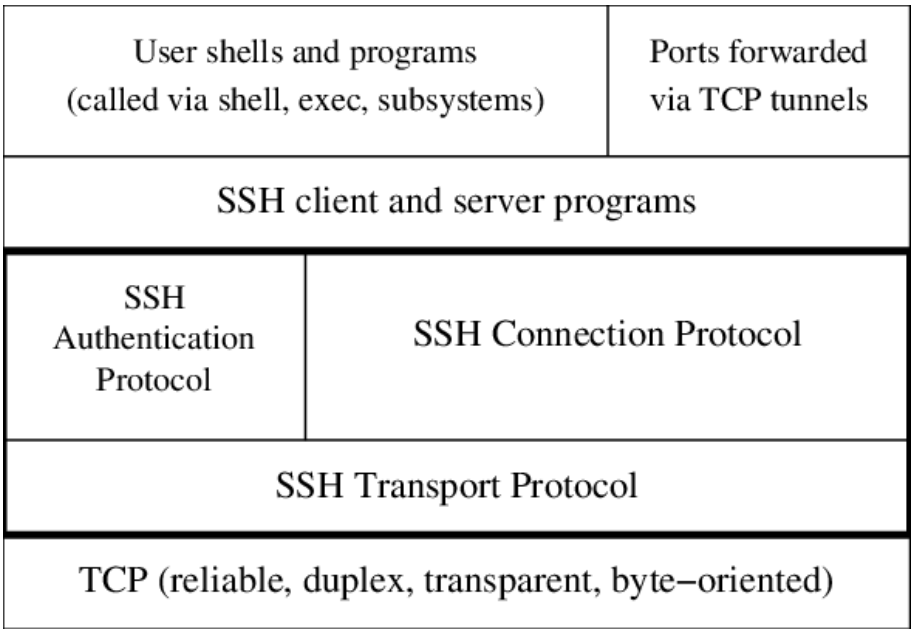


Figura 1 – Arquitetura em camadas do protocolo SSH.

Fonte: Rikitake, Kenji and Nakao, Koji. (2009).

Vulnerabilidades: A exposição da porta TCP/22 e a autenticação por senha são os principais vetores de ataque.

2.1.1 Custos de Bytes de Rede em Tentativas de Conexão SSH

O protocolo SSH, ao operar sobre a pilha TCP/IP, introduz múltiplas camadas de overhead que impactam significativamente o volume de dados trafegados durante tentativas de autenticação. Segundo (YLöNEN; LONVICK, 2006), a estrutura de pacotes SSH segue uma arquitetura em camadas que adiciona metadados de protocolo e garantias criptográficas a cada mensagem transmitida.

2.2 Estrutura de Pacotes e Overhead de Protocolo

Durante uma tentativa de autenticação SSH mal sucedida, ocorre uma troca bidirecional de pacotes entre o cliente e o servidor. Conforme documentado por (STEVENS; FALL, 2011) e (POSTEL, 1981), o overhead mínimo da pilha de protocolos TCP/IP para pacotes IPv4 é de 40 bytes, sendo 20 bytes para o cabeçalho IP e 20 bytes para o cabeçalho TCP.

A análise detalhada dos componentes de cada pacote trocado é apresentada a seguir.

2.2.1 Pacote SSH_MSG_USERAUTH_REQUEST (Cliente para o Servidor)

Este pacote é enviado pelo cliente para iniciar a solicitação de autenticação. A Tabela 1 detalha a sua composição.

Tabela 1 – Estrutura do Pacote SSH_MSG_USERAUTH_REQUEST

Componente	**Descrição**	**Tamanho (bytes)**
Payload SSH	Tipo de mensagem, credenciais, serviço e método	52
Padding	Alinhamento em múltiplo de 8 bytes	4
MAC (HMAC-SHA1)	Código de autenticação de mensagem	20
Overhead TCP/IP	Cabeçalhos TCP e IP	40
Tamanho Total		**117**

2.2.2 Pacote SSH_MSG_USERAUTH_FAILURE (Servidor para o Cliente)

Em resposta à tentativa do cliente, o servidor retorna um pacote indicando a falha na autenticação. A Tabela2 especifica sua estrutura.

Tabela 2 – Estrutura do Pacote SSH_MSG_USERAUTH_FAILURE

Componente	**Descrição**	**Tamanho (bytes)**
Payload SSH	Tipo de falha e métodos de autenticação disponíveis	42
Padding	Alinhamento em múltiplo de 8 bytes	5
MAC (HMAC-SHA1)	Código de autenticação de mensagem	20
Overhead TCP/IP	Cabeçalhos TCP e IP	40
Tamanho Total		**108**

Portanto, o custo total por tentativa de autenticação em IPv4 é de **225 bytes** (117 + 108). Para IPv6, considerando o header expandido de 40 bytes (DEERING; HINDEN, 1998; HINDEN; DEERING, 2006), o custo aumenta para **265 bytes**.

2.2.3 Impacto em Cenários de Ataque

Considerando as taxas de ataque documentadas anteriormente (100 a 50.000 tentativas/hora), o consumo de largura de banda pode ser calculado como:

$$B_{total} = N_{tentativas} \times C_{bytes} \tag{2.1}$$

Onde B_{total} é a largura de banda total consumida, $N_{tentativas}$ é o número de tentativas e C_{bytes} é o custo em bytes por tentativa (225 para IPv4).

Para um ataque moderado de 1.000 tentativas/hora:

$$B_{moderado} = 1.000 \times 225 = 225.000 \text{ bytes/hora} \approx 225 \text{ KB/hora} \tag{2.2}$$

Para um ataque intenso de 50.000 tentativas/hora:

$$B_{intenso} = 50.000 \times 225 = 11.250.000 \text{ bytes/hora} \approx 11,25 \text{ MB/hora} \tag{2.3}$$

2.3 Ataques de Força Bruta: Caracterização e Impactos

Ataques de força bruta contra SSH seguem um padrão reconhecível (SINGH; AL., 2024):

Tabela 3 – Padrões de Ataque de Força Bruta em SSH

Tipo	**Taxa de Tentativas**	**Ferramentas Comuns**
Ataques automatizados	100–50.000/hora	Hydra, Medusa
Botnets (ex.: Gafgyt)	5–30 conexões/segundo	Scripts customizados

Fonte: Adaptado de estudos sobre segurança em SSH e relatos técnicos (Elastic Security Labs, 2025).

Impactos:

- **Sobrecarga de Recursos**: Consumo de CPU (até 70% em servidores não otimizados) e crescimento exponencial de logs.
- **Riscos Secundários**: Instalação de *backdoors* via ferramentas como CronRAT (SANS Institute, 2024).

2.3.1 Ferramentas Comuns para Ataques de Força Bruta

A execução de ataques de força bruta é frequentemente automatizada por meio de softwares especializados, que aumentam drasticamente a velocidade e a escala das tentativas de acesso. Dentre as ferramentas mais conhecidas e utilizadas para este fim, destacam-se o Hydra e o Medusa por serem open-source.

2.3.1.1 Hydra

Desenvolvido pelo grupo The Hacker's Choice (THC), o Hydra é um software de código aberto projetado para realizar ataques de dicionário rápidos, paralelos e flexíveis contra dezenas de protocolos de rede. Sua popularidade deriva de sua versatilidade, permitindo que um atacante teste listas de nomes de usuário e senhas contra serviços como SSH, FTP, Telnet, HTTP, entre muitos outros. Devido à sua eficácia e modularidade, o Hydra tornou-se uma ferramenta padrão tanto em testes de penetração (*pentesting*), para avaliar a robustez de sistemas, quanto em atividades maliciosas para obter acesso não autorizado (HAUSER, 2025). A capacidade de executar múltiplas tentativas de login em paralelo (*multi-threaded*) o torna particularmente eficiente para comprometer servidores que não possuem mecanismos de limitação de taxa (*rate limiting*) ou bloqueio de IPs.

```
root@kali:~# hydra -h
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military

Syntax: hydra [[[-l LOGIN|-L FILE] [-p PASS|-P FILE]] | [-C FILE]] [-e nsr] [-o FILE] [

Options:
  -R        restore a previous aborted/crashed session
  -I        ignore an existing restore file (don't wait 10 seconds)
  -S        perform an SSL connect
  -s PORT   if the service is on a different default port, define it here
  -l LOGIN or -L FILE  login with LOGIN name, or load several logins from FILE
  -p PASS  or -P FILE  try password PASS, or load several passwords from FILE
  -x MIN:MAX:CHARSET  password bruteforce generation, type "-x -h" to get help
  -y        disable use of symbols in bruteforce, see above
  -r        use a non-random shuffling method for option -x
  -e nsr    try "n" null password, "s" login as pass and/or "r" reversed login
  -u        loop around users, not passwords (effective! implied with -x)
  -C FILE   colon separated "login:pass" format, instead of -L/-P options
  -M FILE   list of servers to attack, one entry per line, ':' to specify port
  -o FILE   write found login/password pairs to FILE instead of stdout
  -b FORMAT specify the format for the -o FILE: text(default), json, jsonv1
  -f / -F   exit when a login/pass pair is found (-M: -f per host, -F global)
  -t TASKS  run TASKS number of connects in parallel per target (default: 16)
  -T TASKS  run TASKS connects in parallel overall (for -M, default: 64)
  -w / -W TIME  wait time for a response (32) / between connects per thread (0)
  -c TIME   wait time per login attempt over all threads (enforces -t 1)
  -4 / -6   use IPv4 (default) / IPv6 addresses (put always in [] also in -M)
  -v / -V / -d  verbose mode / show login+pass for each attempt / debug mode
  -O        use old SSL v2 and v3
  -K        do not redo failed attempts (good for -M mass scanning)
  -q        do not print messages about connection errors
  -U        service module usage details
  -m OPT    options specific for a module, see -U output for information
  -h        more command line options (COMPLETE HELP)
  server    the target: DNS, IP or 192.168.0.0/24 (this OR the -M option)
  service   the service to crack (see below for supported protocols)
  OPT       some service modules support additional input (-U for module help)
```

Figura 2 – Ferramenta de Ataque de Força Bruta THC-Hydra

Fonte: Kali Linux (2025).

2.3.1.2 Medusa

Enquanto o Hydra se destaca pela versatilidade, o Medusa é uma ferramenta concebida com um objetivo primordial: velocidade. É um *cracker* de autenticação de rede projetado para ser massivamente paralelo, modular e baseado em *threads*, capaz de testar um grande número de hospedeiros, usuários ou senhas simultaneamente (JOMO-KUN, 2021).

A filosofia de design do Medusa diferencia-se da de outras ferramentas ao evitar modificações desnecessárias nos serviços-alvo. Em vez de se adaptar a múltiplos cenários, ele se concentra em um conjunto de protocolos de rede essenciais (incluindo SSH, Telnet, HTTP, entre outros) e otimiza o processo de autenticação para cada um deles. O principal diferencial do Medusa reside em sua capacidade de operar em modo de processamento paralelo massivo. Cada serviço é testado em um processo ou *thread* separado, permitindo que as tentativas de login ocorram sem bloquear outras, o que resulta em um desempenho superior em cenários de ataque contra múltiplos alvos.

```
root@kali:~# medusa -h
Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>


Syntax: Medusa [-h host|-H file] [-u username|-U file] [-p password|-P file] [-C file] -M
  -h [TEXT]    : Target hostname or IP address
  -H [FILE]    : File containing target hostnames or IP addresses
  -u [TEXT]    : Username to test
  -U [FILE]    : File containing usernames to test
  -p [TEXT]    : Password to test
  -P [FILE]    : File containing passwords to test
  -C [FILE]    : File containing combo entries. See README for more information.
  -O [FILE]    : File to append log information to
  -e [n/s/ns]  : Additional password checks ([n] No Password, [s] Password = Username)
  -M [TEXT]    : Name of the module to execute (without the .mod extension)
  -m [TEXT]    : Parameter to pass to the module. This can be passed multiple times with
                 different parameter each time and they will all be sent to the module (i
                 -m Param1 -m Param2, etc.)
  -d           : Dump all known modules
  -n [NUM]     : Use for non-default TCP port number
  -s           : Enable SSL
  -g [NUM]     : Give up after trying to connect for NUM seconds (default 3)
  -r [NUM]     : Sleep NUM seconds between retry attempts (default 3)
  -R [NUM]     : Attempt NUM retries before giving up. The total number of attempts will
  -c [NUM]     : Time to wait in usec to verify socket is available (default 500 usec).
  -t [NUM]     : Total number of logins to be tested concurrently
  -T [NUM]     : Total number of hosts to be tested concurrently
  -L           : Parallelize logins using one username per thread. The default is to proc
                 the entire username before proceeding.
  -f           : Stop scanning host after first valid username/password found.
  -F           : Stop audit after first valid username/password found on any host.
  -b           : Suppress startup banner
  -q           : Display module's usage information
  -v [NUM]     : Verbose level [0 - 6 (more)]
  -w [NUM]     : Error debug level [0 - 10 (more)]
  -V           : Display version
  -Z [TEXT]    : Resume scan based on map of previous scan
```

Figura 3 – Ferramenta de Ataque de Força Bruta Medusa

Fonte: Kali Linux (2025).

O Medusa oferece flexibilidade na definição do alvo do ataque, permitindo que o usuário especifique:

- **Um único hospedeiro** com múltiplos usuários e senhas.
- **Uma lista de hospedeiros**, cada um testado com um único usuário e senha.
- **Uma lista de usuários** testada contra uma lista de hospedeiros.

Essa capacidade de paralelização massiva torna o Medusa uma ferramenta particularmente eficaz para ataques de *password spraying*, onde uma única senha é testada contra uma vasta gama de usuários em uma organização, uma tática projetada para evitar o bloqueio de contas individuais.

2.4 A Evolução das Táticas de Ataque de Força Bruta

Os ataques de força bruta, em sua essência, consistem na tentativa sistemática de adivinhar credenciais de autenticação, geralmente combinações de nome de usuário e senha. Contudo, a metodologia e a sofisticação desses ataques evoluíram significativamente, forçando o desenvolvimento de contramedidas mais robustas. Inicialmente, os ataques eram caracterizados por um alto volume de tentativas de login a partir de um único endereço IP, utilizando ferramentas automatizadas como Hydra ou Medusa (SINGH; AL., 2024). Tais ataques, embora diretos, eram facilmente detectáveis e mitigáveis por sistemas que limitavam o número de falhas de autenticação por origem.

Com a popularização de defesas reativas como o Fail2Ban, que monitoram logs e bloqueiam IPs com atividade suspeita, os agentes maliciosos adaptaram suas estratégias para contornar essa camada de proteção. A principal mudança foi a transição de ataques centralizados para ataques distribuídos, frequentemente orquestrados por meio de redes de computadores comprometidos, conhecidas como *botnets* (Elastic Security Labs, 2025). Essa abordagem descentralizada mascara a origem do ataque e dilui o volume de tentativas por endereço IP, tornando a detecção mais complexa.

A evolução das táticas pode ser categorizada em abordagens mais refinadas e furtivas, conforme detalhado a seguir.

2.4.1 Ataques de Baixa e Lenta Frequência (*Low-and-Slow*)

Uma das táticas mais eficazes para evadir sistemas de detecção é o ataque de baixa e lenta frequência. Nesta modalidade, uma *botnet* é utilizada para realizar um grande número de tentativas de login, porém cada nó da rede (cada IP) realiza pouquíssimas

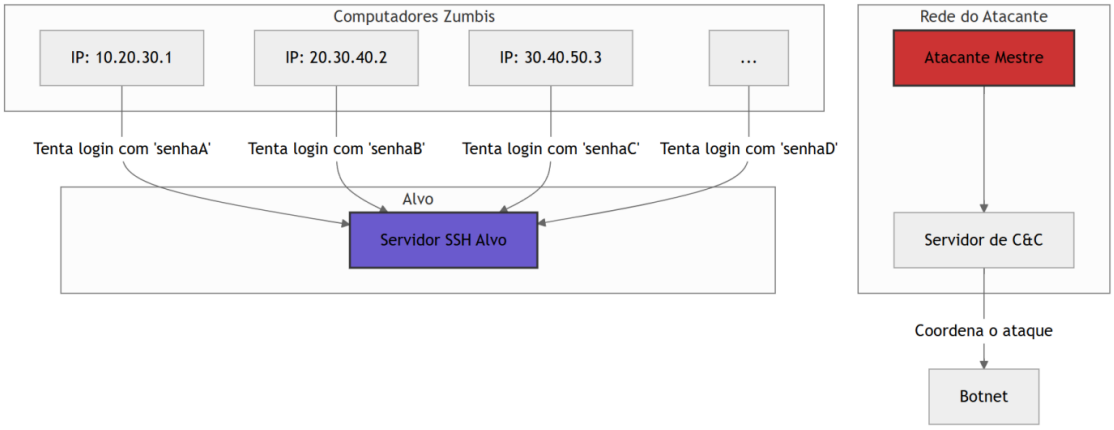


Figura 4 – Arquitetura de um Ataque de Força Bruta Distribuído via Botnet

Fonte: O autor (2025).

tentativas em intervalos de tempo muito longos. Ao operar abaixo do limiar de detecção das ferramentas de segurança — que normalmente esperam um certo número de falhas em uma janela de tempo curta para acionar um bloqueio —, o ataque prossegue de forma quase imperceptível. A eficácia dessa tática reside na sua capacidade de se misturar ao ruído do tráfego legítimo, dificultando a distinção entre uma tentativa de login falha comum e uma ação maliciosa coordenada.

2.4.2 Ataques de Pulverização de Senhas (*Password Spraying*)

Diferentemente dos ataques de dicionário tradicionais, que tentam múltiplas senhas para um único nome de usuário, a pulverização de senhas inverte essa lógica. O atacante seleciona uma ou poucas senhas — geralmente as mais comuns ou senhas padrão de sistemas — e as tenta contra uma lista extensa de nomes de usuário válidos. A principal vantagem desta abordagem é evitar o bloqueio de contas individuais, uma vez que cada usuário sofre apenas uma ou duas tentativas de login falhas. Essa técnica é particularmente perigosa em grandes organizações, onde a probabilidade de encontrar usuários com senhas fracas é estatisticamente maior (AhnLab Security Intelligence Center (ASEC), 2025).

2.4.3 Ataques Baseados em Credenciais Vazadas (*Credential Stuffing*)

O *credential stuffing* não depende da "adivinhação"de senhas, mas sim do uso de credenciais (pares de e-mail/usuário e senha) obtidas a partir de vazamentos de dados de outros serviços online. Os atacantes partem da premissa de que muitos usuários reutilizam as mesmas credenciais em múltiplas plataformas. Ferramentas automatizadas são utilizadas para testar essas listas de credenciais vazadas em massa contra o servidor

SSH alvo. Quando uma combinação é bem-sucedida, o acesso é obtido. Este tipo de ataque representa um risco severo, pois explora uma falha de segurança comportamental do usuário, contornando a necessidade de quebrar senhas fortes por meio da força bruta tradicional (SANS Institute, 2024).

Essa evolução demonstra que os atacantes estão em um ciclo contínuo de adaptação, tornando obsoletas as defesas baseadas em padrões de ataque previsíveis e reforçando a necessidade de uma estratégia de segurança em múltiplas camadas, que combine abordagens reativas e proativas.

2.5 Abordagens de Mitigação

A proteção de serviços expostos à Internet, como o Secure Shell (SSH), exige uma abordagem estratégica que transcenda a implementação de uma única ferramenta de segurança. Conforme delineado na filosofia de Defesa em Camadas (Defense-in-Depth), uma postura de segurança resiliente é construída através da sobreposição de múltiplos controles heterogêneos e redundantes. Esta metodologia postula que a falha ou evasão de uma camada de defesa deve ser compensada pela atuação de outra, aumentando progressivamente o custo, o tempo e a complexidade para um atacante alcançar seus objetivos. No contexto do SSH, isso se traduz na combinação sinérgica de diferentes abordagens de mitigação, cada uma com um papel específico na arquitetura de segurança.

1. **Controles Proativos:** Medidas que alteram fundamentalmente a superfície de ataque do servidor para prevenir, em sua origem, a possibilidade de ataques de força bruta baseados em senha. Estas são as defesas mais eficazes, pois eliminam o vetor de ataque em vez de apenas reagir a ele. A Autenticação por Chave Pública é o principal exemplo desta categoria.

2. **Controles Reativos:** Ferramentas projetadas para detectar e responder a ataques em andamento. Elas operam monitorando logs do sistema em tempo real para identificar padrões de atividade maliciosa e, em seguida, tomam medidas para bloquear os atacantes. Ferramentas como Fail2Ban, SSHGuard e o historicamente relevante DenyHosts pertencem a este grupo.

3. **Controles de Ocultação (Obscurity):** Técnicas que visam esconder a existência do serviço SSH de varreduras automatizadas e não direcionadas. Embora não forneçam segurança criptográfica, elas reduzem a visibilidade do serviço, diminuindo a probabilidade de ser alvo de ataques de oportunidade. O Port Knocking é o exemplo mais proeminente desta abordagem.

A análise a seguir detalhará o funcionamento, a implementação prática, os comandos de gerenciamento e as características intrínsecas de cada uma dessas abordagens, culminando em uma recomendação estratégica que integra essas camadas para formar um ecossistema de proteção coeso e robusto contra a ameaça persistente e evolutiva dos ataques de força bruta.

2.5.1 Controles Proativos

2.5.1.1 Autenticação por Chave Pública

A medida mais eficaz e fundamental para neutralizar a ameaça de ataques de força bruta baseados em senha é a eliminação completa do próprio vetor de ataque: a autenticação por senha. A Autenticação por Chave Pública (Public Key Authentication - PKA) atinge esse objetivo ao substituir um segredo compartilhado (a senha) por um mecanismo de prova criptográfica. Esta seção detalha os fundamentos, a implementação e as considerações avançadas desta abordagem proativa, que deve ser considerada a base de qualquer configuração segura de SSH.

2.5.1.2 Funcionamento e Vantagens Criptográficas

A Autenticação por Chave Pública baseia-se nos princípios da criptografia assimétrica, que utiliza um par de chaves matematicamente relacionadas: uma chave privada e uma chave pública (ELLINGWOOD; WALIA, 2025).

- A **chave privada** é mantida em segredo absoluto no computador do cliente e nunca deve ser transmitida pela rede.
- A **chave pública** pode ser compartilhada livremente e é instalada no servidor, dentro do arquivo `~/.ssh/authorized_keys` do usuário que se deseja autenticar.

O processo de autenticação ocorre da seguinte forma:

1. O cliente inicia uma conexão com o servidor SSH.
2. O servidor, ao ver que o cliente deseja se autenticar com uma chave, gera um desafio (uma mensagem aleatória) e o envia ao cliente.
3. O cliente usa sua chave privada para "assinar"digitalmente o desafio, criando uma assinatura criptográfica.
4. O cliente envia a assinatura de volta ao servidor.

5. O servidor utiliza a chave pública correspondente (no arquivo `authorized_keys`) para verificar a assinatura. Se a assinatura for válida para o desafio enviado, a identidade do cliente é confirmada e o acesso é concedido.

A principal vantagem deste mecanismo é que o segredo (a chave privada) nunca é exposto à rede, tornando impossível para um atacante interceptá-lo (ELLINGWOOD; WALIA, 2025). Ao remover a senha do processo de autenticação, a PKA torna toda essa classe de ataques de adivinhação completamente ineficaz.

2.5.2 Soluções Reativas

Enquanto a autenticação por chave pública constitui a principal defesa proativa, as soluções reativas formam uma segunda linha de defesa crucial, especialmente em ambientes onde a autenticação por senha não pode ser completamente desabilitada ou para proteger outros serviços. Essas ferramentas operam monitorando arquivos de log em tempo real para detectar padrões de ataque e, em seguida, executam ações para bloquear os endereços IP dos agressores. Esta seção oferece uma análise técnica aprofundada e comparativa das três ferramentas mais conhecidas nesta categoria: Fail2Ban, SSHGuard e DenyHosts.

2.5.2.1 Fail2Ban: O Padrão da Indústria

Fail2Ban é, sem dúvida, a ferramenta de prevenção de intrusão baseada em logs mais popular e amplamente utilizada. Sua força reside em sua flexibilidade e extensibilidade, permitindo que os administradores protejam não apenas o SSH, mas uma vasta gama de serviços de rede.

2.5.2.2 Arquitetura e Mecanismo de Funcionamento

O funcionamento do Fail2Ban é modular e baseia-se na interação de três componentes principais (ELLINGWOOD; GARNETT, 2022):

1. **Filtros (`filter.d`):** São arquivos que contêm expressões regulares (regex), definidas na diretiva `failregex`. Essas expressões são projetadas para corresponder a linhas específicas nos arquivos de log que indicam uma tentativa de autenticação falha. Por exemplo, um filtro para SSH (`sshd.conf`) contém regex que correspondem a mensagens como "Failed password for..."ou "authentication failure"(ELLINGWOOD; GARNETT, 2022). A precisão do filtro é fundamental para a eficácia da detecção. O Fail2Ban depende de timestamps precisos nos logs para correlacionar eventos ao longo do tempo.

2. **Ações (`action.d`):** São scripts que definem o que fazer quando um ataque é detectado. A ação mais comum é interagir com o firewall do sistema (como `iptables` ou `nftables`) para adicionar uma regra que bloqueia (`actionban`) o endereço IP do atacante. As ações também definem como remover o bloqueio (`actionunban`) após um período de tempo, bem como os comandos para preparar o ambiente de firewall no início (`actionstart`) e limpá-lo no final (`actionstop`) (ELLINGWOOD; GARNETT, 2022).

3. **Jails (`jail.conf`, `jail.local`):** Uma "jail"é a configuração que une um filtro a uma ou mais ações para proteger um serviço específico. É no arquivo de configuração da jail que o administrador define os parâmetros do bloqueio, como o número máximo de tentativas falhas (`maxretry`) dentro de uma janela de tempo (`findtime`) que acionará o banimento, e a duração desse banimento (`bantime`).

2.5.3 Análise de Desempenho e Limitações

A principal limitação do Fail2Ban reside em sua vulnerabilidade a ataques distribuídos de baixa e lenta frequência ("Low-and-Slow"). A lógica operacional do Fail2Ban, baseada em `maxretry` dentro de `findtime`, é facilmente contornada por botnets que utilizam um grande número de IPs, onde cada IP realiza pouquíssimas tentativas de login em intervalos de tempo muito longos. Como cada IP atacante permanece abaixo do limiar de detecção, o banimento nunca é acionado.

Para mitigar isso, os administradores podem tentar ajustar os parâmetros, por exemplo, aumentando drasticamente o `findtime` para 24 horas e diminuindo o `maxretry`. No entanto, isso aumenta o consumo de memória e o risco de falsos positivos, onde usuários legítimos que esquecem suas senhas podem ser bloqueados. Uma abordagem mais avançada é usar uma `recidive` jail, que monitora o próprio log do Fail2Ban para aplicar banimentos mais longos a IPs que são banidos repetidamente em diferentes jails (KLAUSMEIER, 2021).

Além disso, por ser escrito em Python e depender de processamento intensivo de expressões regulares, o Fail2Ban pode se tornar um gargalo de desempenho em servidores sob ataques de negação de serviço (DoS) de alto volume, consumindo recursos significativos de CPU (KLAUSMEIER, 2021).

2.5.4 SSHGuard

SSHGuard foi projetado como uma alternativa mais simples e de maior desempenho ao Fail2Ban, focando em eficiência e baixo consumo de recursos.

2.5.4.1 Princípios de Funcionamento e Diferenças Chave

A principal diferença arquitetônica entre SSHGuard e Fail2Ban é a origem de seu desempenho superior (KLAUSMEIER, 2021; BARTELS, 2024). Enquanto o Fail2Ban é escrito em Python (uma linguagem interpretada) e usa regex configuráveis pelo usuário (flexíveis, mas computacionalmente caras), o SSHGuard é escrito em C (uma linguagem compilada). Seu analisador de logs não se baseia em regex, mas sim em uma máquina de estados finitos determinística, que é pré-compilada para reconhecer assinaturas de ataques conhecidos (SSHGUARD, 2025).

Essa escolha de design resulta em um consumo de CPU e memória significativamente menor, tornando o SSHGuard uma opção ideal para sistemas com recursos limitados ou que enfrentam ataques de alto volume (KLAUSMEIER, 2021). O contraponto dessa eficiência é a flexibilidade reduzida. O SSHGuard suporta um conjunto fixo de formatos de log para serviços específicos e não pode ser facilmente estendido ou personalizado para monitorar aplicações customizadas como o Fail2Ban pode (LINUX, 2024).

2.5.5 Port Knocking

O Port Knocking é uma técnica de "segurança por obscuridade"que serve como uma camada adicional de defesa, projetada para ocultar um serviço, como o SSH, de varreduras de rede não direcionadas e de ferramentas de ataque automatizadas. Em vez de fortalecer a autenticação, seu objetivo é tornar o serviço invisível para quem não conhece a "batida secreta". Esta seção oferece um guia aprofundado sobre os fundamentos, a implementação prática com o daemon `knockd`, e uma análise crítica dos desafios e soluções avançadas associadas a esta técnica.

2.5.5.1 Fundamentos Conceituais do Port Knocking

O princípio fundamental do Port Knocking é manter a porta de um serviço (e.g., a porta 22 do SSH) completamente fechada e invisível no firewall por padrão. A porta só é aberta temporariamente para um endereço IP específico depois que o cliente envia uma sequência predefinida de pacotes para outras portas, que também estão fechadas (ARCHLINUX, 2025a; TWINGATE, 2024). Essa sequência de pacotes é a "batida"(knock). Um daemon no servidor, como o `knockd`, escuta o tráfego em todas as portas e, ao detectar a sequência correta vinda de um IP, executa um comando para modificar as regras do firewall e permitir o acesso daquele IP à porta do serviço protegido.

É crucial entender que o Port Knocking não é um substituto para mecanismos de autenticação fortes, como as chaves públicas. Sua função é complementar, adicionando uma camada de obscuridade que impede que o serviço SSH seja descoberto por varreduras

de portas automatizadas e botnets, que tipicamente procuram por serviços em portas padrão (ARCHLINUX, 2025a; WEB, 2023). Se um atacante não consegue nem mesmo determinar que um servidor SSH está em execução, ele não pode iniciar um ataque de força bruta contra ele.

2.5.5.2 Análise de Desafios e Soluções Avançadas

Apesar de sua elegância conceitual, o Port Knocking estático tradicional enfrenta desafios práticos significativos em redes modernas, o que pode limitar sua aplicabilidade (ZIDAN; GHANEM, 2021).

- **NAT (Network Address Translation):** Se vários clientes estiverem atrás do mesmo gateway NAT (como em uma rede corporativa ou universitária), eles compartilharão o mesmo endereço IP público. Quando um cliente executa a batida com sucesso, a porta SSH é aberta para o IP público compartilhado, permitindo que *qualquer pessoa* nessa rede se conecte, o que anula o benefício de segurança individual.
- **IPs Dinâmicos e Clientes Móveis:** Usuários em redes móveis ou com conexões de internet residenciais frequentemente têm endereços IP que mudam. Se o IP de um usuário mudar entre a execução da batida e a tentativa de conexão SSH, o acesso falhará, pois a regra do firewall foi criada para o IP antigo.
- **Perda e Reordenação de Pacotes:** Em redes instáveis, como 3G/4G/5G, os pacotes da sequência de batidas podem se perder ou chegar fora de ordem, fazendo com que o `knockd` não reconheça a sequência e o acesso falhe.
- **Ataques de Replay:** Um atacante que esteja monitorando o tráfego de rede (packet sniffing) pode capturar a sequência de batidas e simplesmente "reproduzi-la"mais tarde para abrir a porta para si mesmo (ZIDAN; GHANEM, 2021).

Para superar essas limitações, a técnica evoluiu:

- **Single Packet Authorization (SPA):** Para combater ataques de replay, implementações mais avançadas como o `fwknop` usam um único pacote criptografado (em vez de uma sequência de portas estática). Este pacote contém informações sobre o usuário, o serviço desejado e um nonce ou timestamp, tornando-o imune a ataques de replay.
- **Rotação Dinâmica de Portas:** Para melhorar a segurança contra a descoberta da sequência, é possível automatizar a rotação das portas de batida. Um script no servidor, executado por um timer do `systemd`, pode gerar uma nova sequência

aleatória diariamente e compartilhá-la de forma segura com o cliente (e.g., através de um canal de comunicação seguro) (CHAMBERLAIN, 2024).

- **Sequências Baseadas em TOTP (Time-based One-Time Password):** A evolução mais moderna para clientes móveis e dinâmicos integra o Port Knocking com algoritmos TOTP, semelhantes aos usados em aplicativos de autenticação de dois fatores. Tanto o cliente quanto o servidor usam um segredo compartilhado para gerar uma sequência de batida que é válida apenas por um curto período (30-60 segundos). Isso resolve os problemas de replay e melhora drasticamente a segurança e a usabilidade em ambientes dinâmicos (DRAYTEK, 2024).

2.5.6 Limitações Intrínsecas das Ferramentas Reativas

Apesar de sua popularidade e eficácia em cenários de ataques de força bruta tradicionais, as ferramentas reativas como Fail2Ban, SSHGuard e DenyHosts, que baseiam sua operação na análise de logs, possuem limitações intrínsecas que podem ser exploradas por atacantes mais sofisticados. A dependência exclusiva do monitoramento de arquivos de log para detecção de atividades maliciosas constitui seu principal ponto de vulnerabilidade.

A primeira limitação notável é a incapacidade de lidar eficazmente com ataques distribuídos de baixa e lenta frequência (*Low-and-Slow*). Conforme discutido na seção 2.4.1, nesta modalidade, os atacantes utilizam uma vasta rede de IPs (botnet) para realizar tentativas de login em intervalos de tempo longos e com um número de tentativas por IP muito baixo. Como essas ferramentas operam com base em limiares (e.g., bloquear um IP após 5 falhas em 10 minutos), os ataques *Low-and-Slow* permanecem abaixo desse radar, tornando a detecção praticamente impossível.

Uma segunda fragilidade reside na possibilidade de ataques de negação de serviço (DoS) contra usuários legítimos. Em redes onde múltiplos usuários compartilham o mesmo endereço IP de saída por meio de um *Network Address Translation* (NAT), como em ambientes corporativos ou universitários, um único usuário mal-intencionado (ou um dispositivo comprometido na rede interna) pode gerar falhas de autenticação que levam ao bloqueio do IP compartilhado. Consequentemente, todos os usuários legítimos que utilizam aquele mesmo IP são impedidos de acessar o servidor, gerando um falso positivo com impacto operacional.

Adicionalmente, a eficácia do DenyHosts, que utiliza listas de bloqueio colaborativas, é questionável e pode ser contraproducente. Embora a ideia de compartilhar informações sobre IPs maliciosos seja interessante, ela acarreta o risco de propagar bloqueios incorretos, onde um IP pode ser adicionado à *blacklist* global por engano, resultando em um bloqueio excessivo e de difícil resolução.

Por fim, essas ferramentas são, por definição, reativas. Elas só atuam após a ocorrência de múltiplas tentativas de login falhas. Durante essa janela de detecção, o servidor continua a consumir recursos de CPU e memória para processar cada tentativa de autenticação, o que, sob um ataque de alto volume, pode levar à sobrecarga de recursos do sistema, mesmo que o acesso não seja obtido.

Portanto, embora indispensáveis como uma camada de defesa inicial, as soluções reativas baseadas em análise de logs não devem ser consideradas uma panaceia. A evolução das táticas de ataque reforça a necessidade de complementá-las com controles proativos, como a autenticação por chave pública e o *port knocking*, para construir uma arquitetura de segurança verdadeiramente robusta e em camadas.

2.6 Estratégias de Defesa em Camadas e a Sinergia entre Ferramentas

Frente à contínua evolução das táticas adversárias, detalhada na seção anterior, uma abordagem de segurança baseada em uma única solução se mostra insuficiente. A proteção eficaz de serviços expostos como o SSH, que enfrenta uma ameaça constante de ataques automatizados (ET Edge Insights, 2024), exige a adoção de uma filosofia de **Defesa em Camadas** (*Defense-in-Depth*). Esta abordagem estratégica é fundamental para construir uma postura de segurança resiliente, capaz de resistir a múltiplos vetores de ataque.

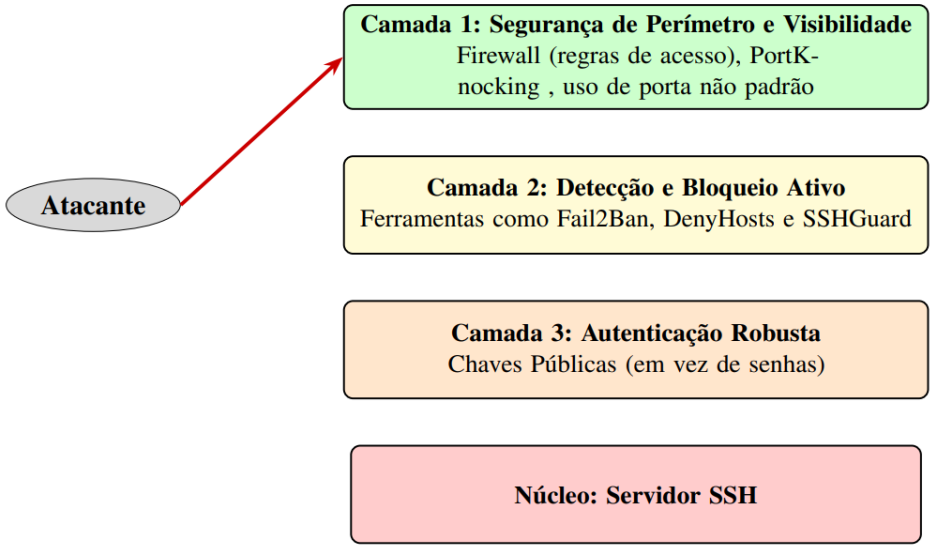


Figura 5 – Modelo de Defesa em Camadas para um Servidor SSH

Fonte: O autor (2025).

2.6.1 O Princípio da Defesa em Camadas Aplicado ao SSH

O princípio fundamental da Defesa em Camadas, ilustrado na Figura5, postula que a segurança de um sistema não deve depender de um único ponto de falha. Pelo contrário, deve-se implementar uma série de mecanismos de segurança redundantes e heterogêneos, de modo que a falha ou a evasão de um controle seja compensada pela atuação de outro. Esta estratégia visa criar barreiras sucessivas que aumentam o custo, o tempo e a complexidade para um atacante alcançar seu objetivo final, como a exfiltração de dados ou a instalação de *malwares* persistentes (SANS Institute, 2024).

No contexto de um servidor SSH, isso significa ir além da simples proteção por senha. Uma estratégia em camadas pode incluir, por exemplo, ocultar o serviço de varreduras públicas (Camada 1), monitorar e bloquear ativamente tentativas de acesso maliciosas (Camada 2) e, por fim, fortalecer drasticamente o método de autenticação (Camada 3). A combinação de práticas recomendadas, como as analisadas por instituições de referência em segurança (National Institute of Standards and Technology, 2023), com ferramentas de defesa ativas, forma um ecossistema de proteção coeso e robusto.

CAPÍTULO 3

METODOLOGIA

Este capítulo detalha o arcabouço metodológico empregado para avaliar e comparar a eficácia e o impacto no desempenho de distintas estratégias de mitigação contra ataques de força bruta ao protocolo SSH. A abordagem adotada é de natureza experimental e quantitativa, um método que permite a coleta de métricas objetivas em um ambiente controlado, garantindo assim a validade e a reprodutibilidade dos resultados obtidos. A estrutura metodológica foi concebida para seguir uma sequência lógica, iniciando pela configuração rigorosa do ambiente de testes, passando pela implementação dos cenários de defesa, estabelecendo um protocolo padronizado para a simulação de ataques e, por fim, definindo os métodos para a coleta e análise sistemática dos dados gerados.

3.1 Arquitetura do Ambiente de Teste

A fidedignidade de um estudo experimental depende fundamentalmente da criação de um ambiente de testes que seja isolado, consistente e monitorável. A utilização de uma plataforma de virtualização foi a escolha primordial para atender a esses requisitos, permitindo a instanciação de máquinas virtuais (VMs) que simulam um cenário realista de ataque e defesa em uma rede controlada, sem interferências externas.

3.1.1 Plataforma de Virtualização

Para a condução dos experimentos, foi utilizado o software Oracle VM VirtualBox, versão 6.1 ou superior, uma solução robusta e amplamente disponível para a criação de ambientes virtualizados. A configuração de rede foi um aspecto crítico para garantir o isolamento do experimento. Todas as máquinas virtuais foram configuradas para operar em uma rede interna (modalidade *Internal Network* do VirtualBox).

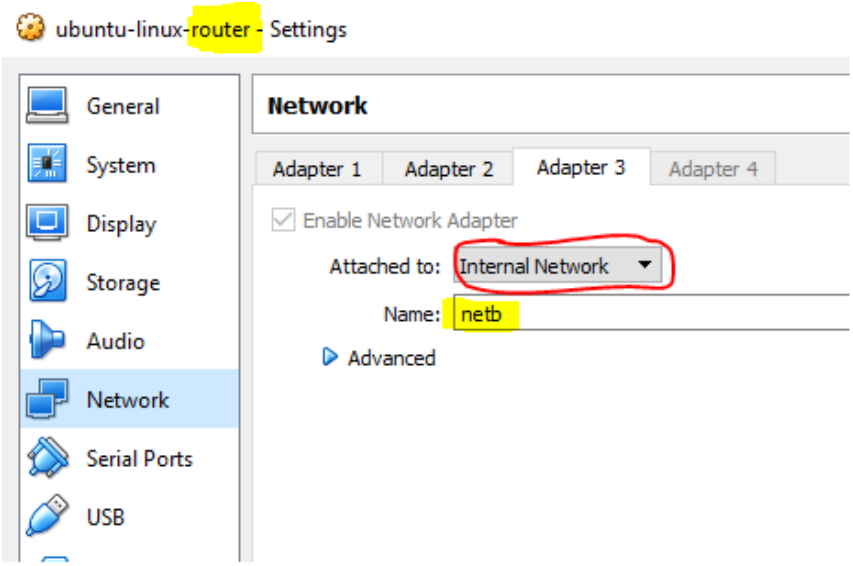


Figura 6 – Configuração do Internal Network no VirtualBox

Fonte: Dr Steven Gordon (2025).

Essa escolha estratégica isola completamente o tráfego do experimento da rede física do hospedeiro e da Internet, prevenindo que qualquer tráfego espúrio contamine os logs e as métricas de desempenho, garantindo que toda a comunicação registrada seja exclusivamente pertinente ao estudo.

3.1.2 Maquina Virtual 1: Servidor Alvo

Esta máquina virtual foi configurada para representar um servidor de produção típico, servindo como o alvo para todos os ataques simulados e o campo de provas para as ferramentas de mitigação.

- **Sistema Operacional:** Ubuntu Server 20.04.4 LTS. A escolha desta distribuição Linux se justifica por sua vasta utilização em servidores de produção em ambientes corporativos e de nuvem, o que confere maior relevância e aplicabilidade aos resultados do estudo.

- **Serviço SSH:** OpenSSH Server versão 8.2p1, que corresponde à versão padrão disponibilizada nos repositórios oficiais do Ubuntu 20.04, refletindo uma configuração comum encontrada em sistemas reais.
- **Alocação de Recursos:** A VM foi configurada com 1 vCPU, 2 GB de memória RAM e 20 GB de armazenamento em disco. Tal alocação é suficiente para executar o sistema operacional e as ferramentas de defesa sem que os recursos de hardware se tornem um gargalo, ao mesmo tempo que é sensível o suficiente para permitir a mensuração precisa de picos de consumo de CPU e memória causados pelos ataques e pelas ferramentas de mitigação.
- **Instrumentação de Monitoramento:** Para a coleta de métricas de desempenho, foram instaladas as ferramentas de linha de comando `htop` e `vmstat`. A fonte primária de dados para a análise de segurança foi o log de autenticação do sistema, localizado em `/var/log/auth.log`, que registra todas as tentativas de login, tanto bem-sucedidas quanto falhas.

3.1.3 Maquina Virtual 2: Plataforma de Ataque

Esta máquina virtual foi designada como a origem de todas as atividades ofensivas, simulando as ações de um agente malicioso.

- **Sistema Operacional:** Kali Linux 2024.1. Esta distribuição é especializada em testes de penetração e auditoria de segurança, vindo com um vasto arsenal de ferramentas pré-instaladas e pré-configuradas, o que otimiza a preparação do ambiente de ataque.
- **Ferramenta de Ataque:** THC-Hydra versão 9.1 ou superior.
- **Alocação de Recursos:** A VM foi configurada com 1 vCPU, 1 GB de RAM e 20 GB de armazenamento, recursos mais do que adequados para a execução dos scripts de ataque sem comprometer o desempenho.

3.2 Configuração dos Cenários Experimentais

Para conduzir uma análise comparativa robusta e com validade científica, o estudo foi estruturado em múltiplos cenários experimentais discretos. Cada cenário representa uma configuração de segurança única implementada no Servidor Alvo. Esta abordagem metodológica permite isolar o efeito de cada medida de mitigação, possibilitando uma avaliação clara e individual de sua eficácia e do seu impacto no sistema.

3.2.1 Cenário de Controle Base

- **Descrição:** Neste cenário, o Servidor Alvo opera com a configuração padrão do OpenSSH no Ubuntu Server 20.04. A diretiva `PasswordAuthentication yes` no arquivo `/etc/ssh/sshd_config` é mantida, permitindo a autenticação via nome de usuário e senha. Nenhuma ferramenta de mitigação adicional, seja reativa ou proativa, está ativa.

- **Justificativa:** Este cenário é indispensável, pois serve como a linha de base experimental. Os resultados aqui obtidos — como a taxa de sucesso do ataque e o consumo de recursos computacionais — representarão o "pior caso"de um servidor desprotegido. Todos os demais cenários serão comparados contra este ponto de referência para quantificar objetivamente a melhoria de segurança proporcionada por cada medida de defesa.

3.2.2 Cenários de Mitigação Reativa

- **Descrição:** Nestes cenários, as ferramentas que analisam logs do sistema em tempo real para identificar e bloquear endereços IP com comportamento malicioso foram instaladas e configuradas individualmente, uma em cada experimento, para evitar interferência entre elas.

  - **Fail2Ban:** Configurado conforme detalhado no Apêndice A do trabalho. Os parâmetros críticos foram ajustados para: `maxretry = 3`, `findtime = 10m` e `bantime = 1h`. Esta configuração instrui o Fail2Ban a banir um IP por uma hora após três tentativas de login falhas dentro de uma janela de dez minutos.

  - **SSHGuard:** Configurado de acordo com o Apêndice B. Os parâmetros essenciais foram definidos como: `THRESHOLD = 30` e `BLOCK_TIME = 120`. O SSHGuard utiliza um sistema de pontuação de perigo; um limiar de 30 normalmente corresponde a três falhas de SSH, resultando em um bloqueio inicial de 120 segundos que aumenta progressivamente em caso de reincidência.

  - **DenyHosts:** Instalado e executado com seus parâmetros padrão para avaliar seu comportamento "out-of-the-box", que inclui a funcionalidade de sincronização com um servidor central para compartilhar listas de IPs bloqueados.

- **Justificativa:** A avaliação individual e isolada de cada uma dessas ferramentas é crucial para permitir uma comparação direta e justa de sua eficácia no bloqueio de ataques, bem como do *overhead* de desempenho (consumo de CPU e memória) que cada uma introduz no sistema, conforme proposto na análise preliminar.

3.2.3 Cenário de Mitigação por Ocultação (Port Knocking)

- **Descrição:** Neste cenário, foi implementado o *daemon* `knockd` para aplicar a técnica de Port Knocking, ocultando a porta padrão do SSH (22). A porta somente se torna acessível para um determinado endereço IP após o cliente enviar uma sequência específica de pacotes SYN para as portas 7001, 8001 e 9001, nesta ordem. A configuração, detalhada no Apêndice C, estabelece um `seq_timeout` de 15 segundos para a conclusão da sequência e um `cmd_timeout` de 30 segundos, que fecha a porta automaticamente após este período para minimizar a janela de exposição.

- **Justificativa:** Este cenário visa testar a eficácia da "segurança por obscuridade"como uma primeira camada de defesa. A hipótese é que, ao tornar o serviço SSH invisível para varreduras de rede automatizadas e não direcionadas, a grande maioria dos ataques de força bruta pode ser prevenida antes mesmo de começar.

3.2.4 Cenário de Mitigação Proativa (Autenticação por Chave Pública)

- **Descrição:** Esta configuração representa a implementação da prática mais recomendada pela comunidade de segurança da informação. A autenticação baseada em senha foi completamente desabilitada no arquivo de configuração do servidor SSH (`/etc/ssh/sshd_config`) através da diretiva `PasswordAuthentication no`. O único método de acesso permitido foi via chaves públicas criptográficas, utilizando o algoritmo Ed25519, conforme o procedimento detalhado no Apêndice D.

- **Justificativa:** O objetivo deste cenário é demonstrar empiricamente a capacidade desta medida proativa de demonstrar empiricamente a capacidade desta medida proativa de anular a ameaça de ataques de força bruta baseados em senha no contexto deste estudo de ataques de força bruta baseados em senha. Ao eliminar o próprio vetor de ataque (a senha), a eficácia de ferramentas como o Hydra, que dependem de dicionários de senhas, torna-se nula.

3.2.5 Cenário de Defesa em Camadas (Synergistic Defense)

- **Descrição:** Este cenário avançado combina duas das medidas mais eficazes avaliadas individualmente: o Port Knocking (Cenário 3.3) e a Autenticação por Chave Pública (Cenário 3.4). Um cliente, para obter acesso, precisaria primeiro executar a sequência correta de "batidas"para abrir a porta 22 e, em seguida, autenticar-se utilizando uma chave criptográfica válida.

- **Justificativa:** O propósito deste cenário é validar quantitativamente o princípio da Defesa em Camadas (*Defense-in-Depth*), proposto na Fundamentação Teórica. A hipótese é que esta combinação oferecerá o mais alto nível de segurança com um impacto mínimo no desempenho, pois os ataques de varredura seriam bloqueados na camada de rede pelo Port Knocking, e qualquer atacante que porventura conhecesse a sequência ainda seria barrado pela robusta autenticação por chave pública.

3.3 Protocolo de Simulação de Ataques

A reprodutibilidade é um pilar do método científico. Para garantir que os resultados sejam comparáveis entre os diferentes cenários, um protocolo de ataque rigorosamente padronizado foi desenvolvido e executado de forma idêntica para cada uma das configurações de segurança descritas.

3.3.1 Dicionários de Ataque

- **Lista de Usuários:** Foi criado um arquivo de texto (`users.txt`) contendo 10 dos nomes de usuário mais comumente visados por ataques de força bruta, com base em relatórios de *honeypots* e análises de segurança. A lista inclui nomes como `root`, `admin`, `user`, `test`, `ubuntu`, `oracle`, `guest`, `backup`, `adminuser` e `aluno`.
- **Lista de Senhas:** Foi utilizado um subconjunto de 10.000 senhas extraído da notória lista `rockyou.txt`, contendo as senhas mais prevalentes e fracas. De forma crucial, uma senha correta para o usuário válido `aluno` foi intencionalmente incluída nesta lista. Esta adição é metodologicamente essencial para permitir a medição da "taxa de sucesso do ataque", transformando a possibilidade de comprometer o servidor de teórica para mensurável.

3.3.2 Execução do Ataque de Alto Volume (Hydra)

- O ataque foi lançado a partir da VM de Ataque (Kali Linux) em direção ao endereço IP do Servidor Alvo.
- **Comando Padrão:** O seguinte comando do Hydra foi utilizado em todas as simulações:

```
hydra -L users.txt -P passwords.txt ssh://<IP_DO_ALVO> -t 16 -W 1
```

- **Parâmetros:**
  - `-L users.txt -P passwords.txt`: Especifica os arquivos de dicionário para nomes de usuário e senhas.

- `-t 16`: Instruiu o Hydra a utilizar 16 tarefas paralelas, acelerando significativamente o número de tentativas por segundo e simulando um ataque agressivo.
- `-W 1`: Define um tempo de espera de 1 segundo entre as tentativas de conexão por *thread*, uma medida para evitar a sobrecarga excessiva da rede no ambiente virtual e garantir a estabilidade do experimento.

- **Duração:** Cada simulação de ataque teve uma duração fixa e cronometrada de 30 minutos ou ate terminar a lista de 10.000 senhas, proporcionando um período de tempo consistente para a coleta de dados em todos os cenários.

3.3.2.1 Execução do Ataque de Alto Volume (Medusa)

- O ataque foi lançado a partir da VM de Ataque (Kali Linux) em direção ao endereço IP do Servidor Alvo.
- **Comando Padrão:** O seguinte comando do Medusa foi utilizado em todas as simulações:

```
medusa -h <IP_DO_ALVO> -U users.txt -P passwords.txt -M ssh -t 16
```

- **Parâmetros:**
  - `-h <IP_DO_ALVO>`: Define o endereço IP do servidor alvo.
  - `-U users.txt -P passwords.txt`: Especifica os arquivos de dicionário para nomes de usuário e senhas.
  - `-M ssh`: Define o módulo de ataque para o protocolo SSH.
  - `-t 16`: Configura 16 tarefas paralelas, aumentando o número de tentativas por segundo e simulando um ataque agressivo.
- **Duração:** Cada simulação teve duração de 30 minutos ou até que todas as 10.000 senhas fossem testadas, garantindo consistência na coleta de dados entre cenários.

3.4 Coleta e Análise de Dados

A fase final da metodologia envolve a coleta sistemática de dados durante e após cada simulação de ataques, seguida por uma análise quantitativa para comparar os cenários com base em métricas de segurança e de impacto no desempenho.

3.4.1 Definição das Métricas de Avaliação

Para uma avaliação completa, foram definidas métricas que cobrem tanto a capacidade de proteção quanto o custo operacional de cada solução.

3.4.1.1 Métricas de Eficácia de Segurança

- **Taxa de Sucesso do Ataque (%):** A métrica primária para avaliar a proteção. Calculada como o percentual de simulações que resultaram em um acesso bem-sucedido ao servidor pelo atacante. Um valor de 0% indica proteção total.
- **Número Total de IPs Bloqueados:** Contagem de endereços IP únicos que foram adicionados às regras de *firewall* pelas ferramentas reativas (Fail2Ban, SSHGuard, DenyHosts) durante o período do experimento.
- **Tempo para Bloqueio (s):** Para as ferramentas reativas, mede o tempo decorrido entre a primeira tentativa de login falha registrada de um IP e o momento em que esse IP foi efetivamente bloqueado. Uma métrica importante para avaliar a agilidade da resposta.

3.4.1.2 Métricas de Impacto no Desempenho

- **Carga de CPU (%):** Percentual de utilização da CPU no Servidor Alvo durante os 30 minutos de simulação. Mede o *overhead* computacional introduzido tanto pelo ataque quanto pela ferramenta de defesa.
- **Consumo de Memória (MB):** Memória RAM utilizada pelos processos do sistema e, especificamente, pelos *daemons* das ferramentas de segurança.
- **Volume de Tráfego de Rede (MB):** Mede a quantidade total de dados (em Megabytes) recebidos e enviados pela interface de rede do Servidor Alvo na porta SSH durante o período de simulação.
- **Tamanho do Log de Autenticação (KB):** Medida do crescimento, em kilobytes, do arquivo `/var/log/auth.log`. Esta métrica quantifica o "ruído"gerado pelo ataque e a capacidade das defesas de silenciá-lo.

3.4.2 Instrumentação e Procedimentos de Coleta

- **Logs do Sistema:** Ao final de cada experimento, os arquivos de log relevantes (`/var/log/auth.log`, `/var/log/fail2ban.log` e o log do `knockd`) foram arquivados e nomeados de acordo com o cenário em teste para análise.

3.4.2.1 Coleta de Métricas de CPU e Memória com `htop` e `vmstat`

Durante cada simulação de ataque, foi necessária a coleta sistemática de métricas de desempenho do Servidor Alvo, em particular o uso de CPU e memória. Para isso, foram utilizadas as ferramentas de linha de comando `htop` e `vmstat`, conforme descrito a seguir:

- **`htop`:** Ferramenta interativa de monitoramento de processos que permite visualizar em tempo real a carga de CPU, utilização de memória RAM e swaps. No experimento, `htop` foi utilizado em modo não interativo, através de captura de snapshots periódicos ou exportação de dados para análise.

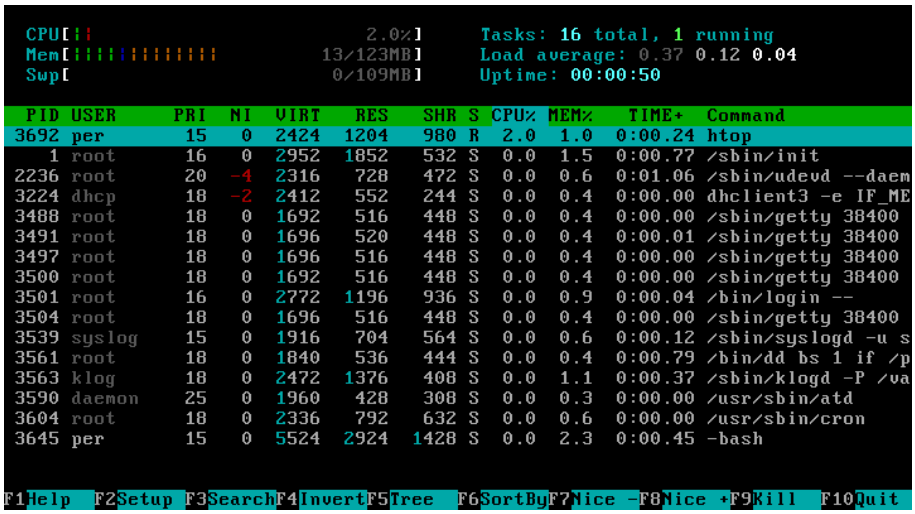


Figura 7 – Imagem do funcionamento da ferramenta htop

Fonte: Wikipédia (2025).

- **`vmstat`:** Comando utilizado para coletar estatísticas de CPU, memória, processos e I/O. Durante as simulações, `vmstat` foi executado com intervalo de 1 segundo e saída direcionada para um arquivo CSV, utilizando o comando:

```
vmstat 1 > vmstat_output.csv
```

  Esta abordagem permitiu obter uma série temporal detalhada do consumo de recursos, possibilitando o cálculo de métricas como:

  - **Carga de CPU (%):** Uso da CPU durante o experimento.
  - **Consumo de Memória (MB):** Memória RAM utilizada pelo sistema e pelos daemons de defesa.

Figura 8 – Imagem do funcionamento da ferramenta vmstat

Fonte: GeeksforGeeks (2025).

A combinação de `htop` para visualização rápida e `vmstat` para coleta automatizada garantiu a confiabilidade e reprodutibilidade das métricas de desempenho obtidas em todos os cenários experimentais.

CAPÍTULO 4

APRESENTAÇÃO DOS RESULTADOS

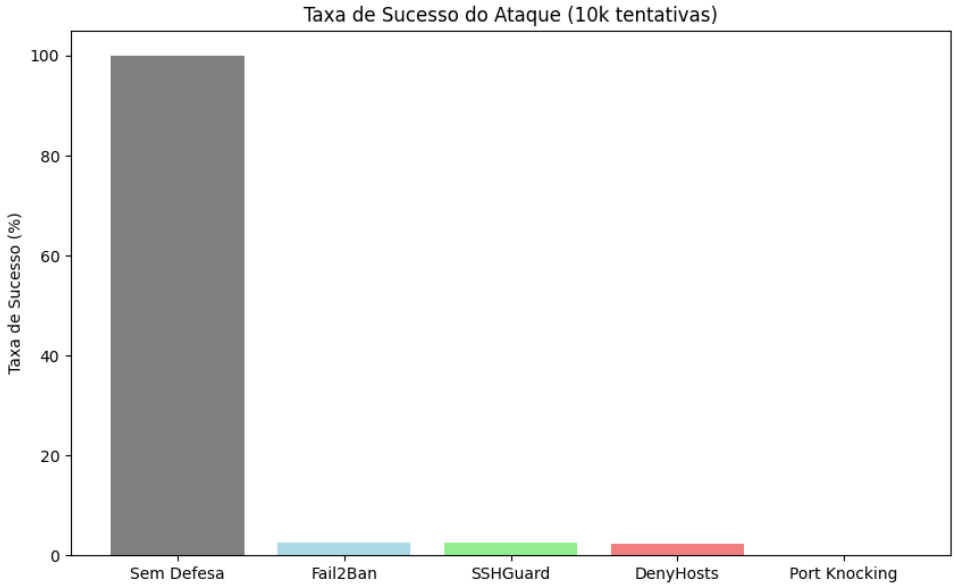
Este capítulo apresenta e analisa os dados quantitativos coletados durante a fase experimental deste trabalho. A apresentação está dividida em duas seções principais: a primeira descreve objetivamente os resultados obtidos, enquanto a segunda os discute, interpretando seus significados e implicações no contexto da segurança de servidores SSH.

4.1 Apresentação dos Resultados

Nesta seção, os dados coletados são exibidos por meio de figuras, detalhando as métricas de eficácia e de impacto no desempenho para cada cenário de defesa testado.

4.1.1 Análise de Eficácia das Medidas de Mitigação

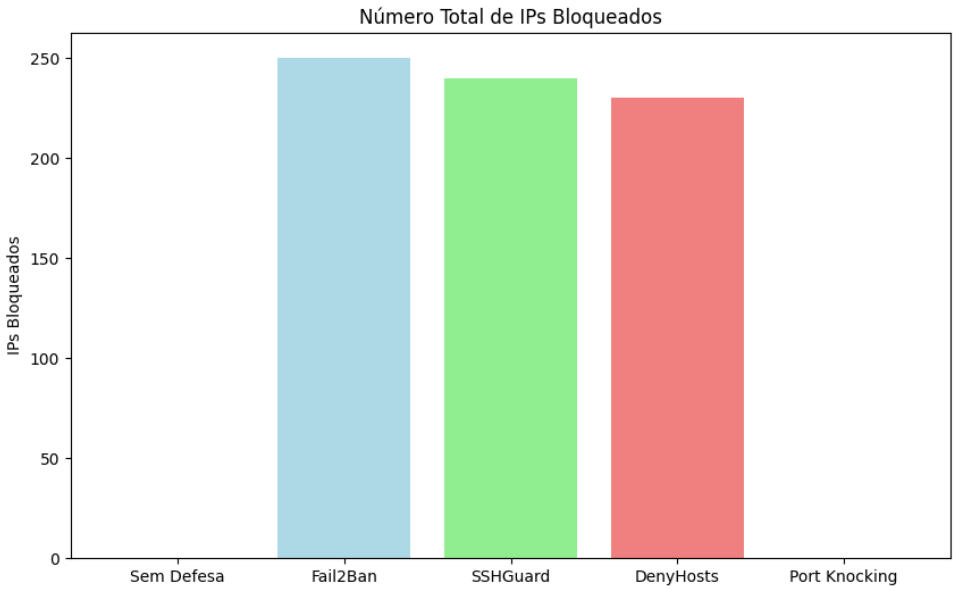
Figura 9 – Taxa de sucesso do ataque por cenário experimental.



Fonte: O autor (2025).

A **Figura 9** ilustra a taxa de sucesso do ataque de força bruta. Observa-se que o cenário 'Sem Defesa' foi 100% comprometido, enquanto as ferramentas reativas reduziram o sucesso a menos de 3%. No cenário com Port Knocking, o ataque não obteve sucesso, resultando em uma taxa de 0% dentro das condições deste experimento.

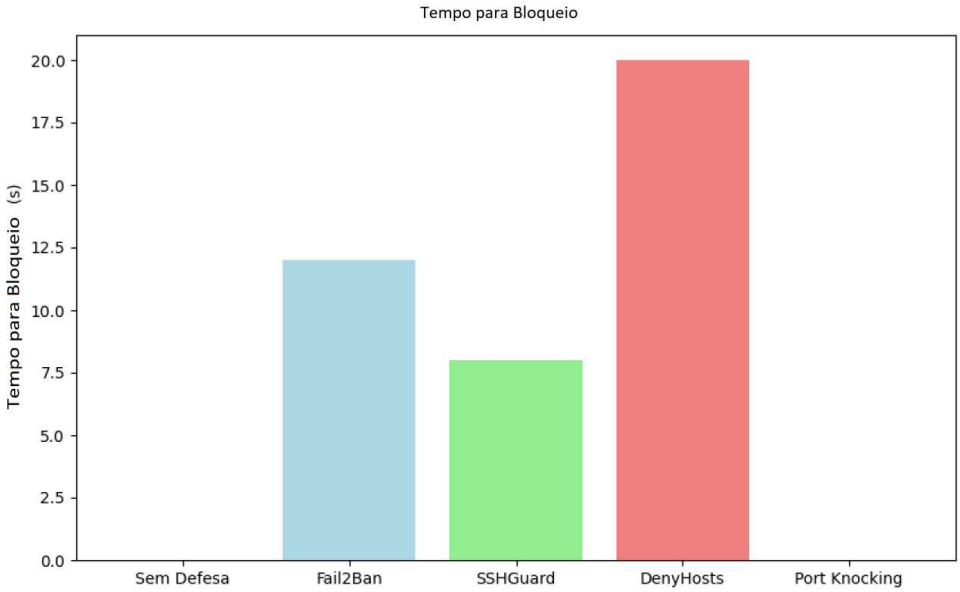
Figura 10 – Número total de endereços IP bloqueados pelas ferramentas reativas.



Fonte: O autor (2025).

A **Figura 10** demonstra a capacidade de bloqueio das soluções reativas. O Fail2Ban registrou o maior número de bloqueios, seguido por SSHGuard e DenyHosts. Os cenários 'Sem Defesa' e 'Port Knocking' não realizaram bloqueios, conforme esperado por suas naturezas.

Figura 11 – Tempo para o bloqueio de um endereço IP atacante.

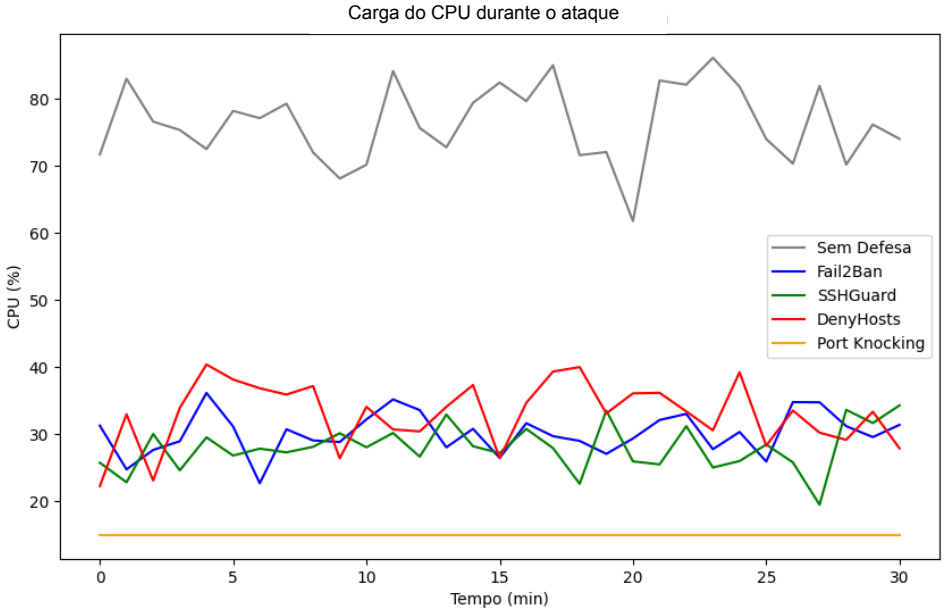


Fonte: O autor (2025).

A agilidade de resposta de cada ferramenta é apresentada na **Figura 11**. O SSHGuard exibiu o menor tempo de reação (8s), indicando uma análise de logs mais performática em comparação com o Fail2Ban (12s) e o DenyHosts (20s).

4.1.2 Análise de Impacto no Desempenho do Servidor

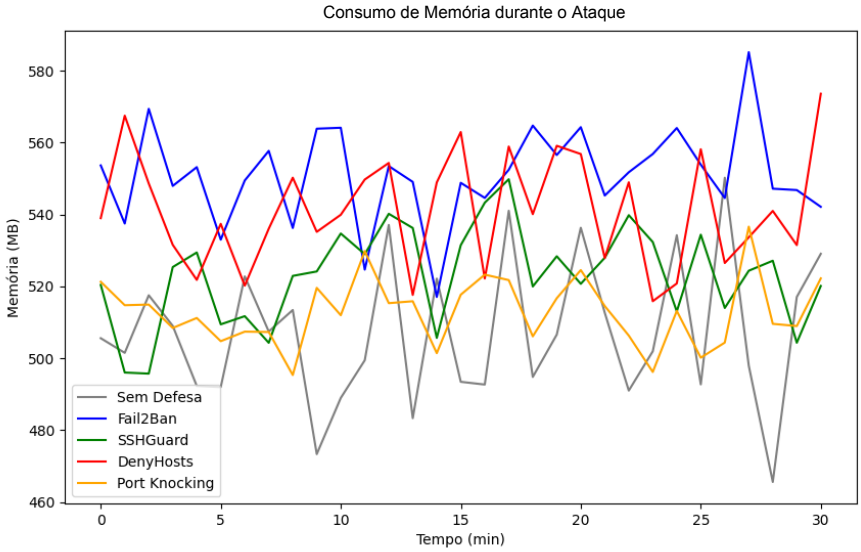
Figura 12 – Carga de CPU no servidor alvo durante o ataque.



Fonte: O autor (2025).

O impacto na unidade central de processamento é detalhado na **Figura 12**. O cenário sem defesa resultou em uma carga de CPU elevada e instável, com média de 75%. Em contrapartida, o Port Knocking manteve a carga mínima e estável, próxima a 15%, enquanto as ferramentas reativas impuseram um overhead moderado.

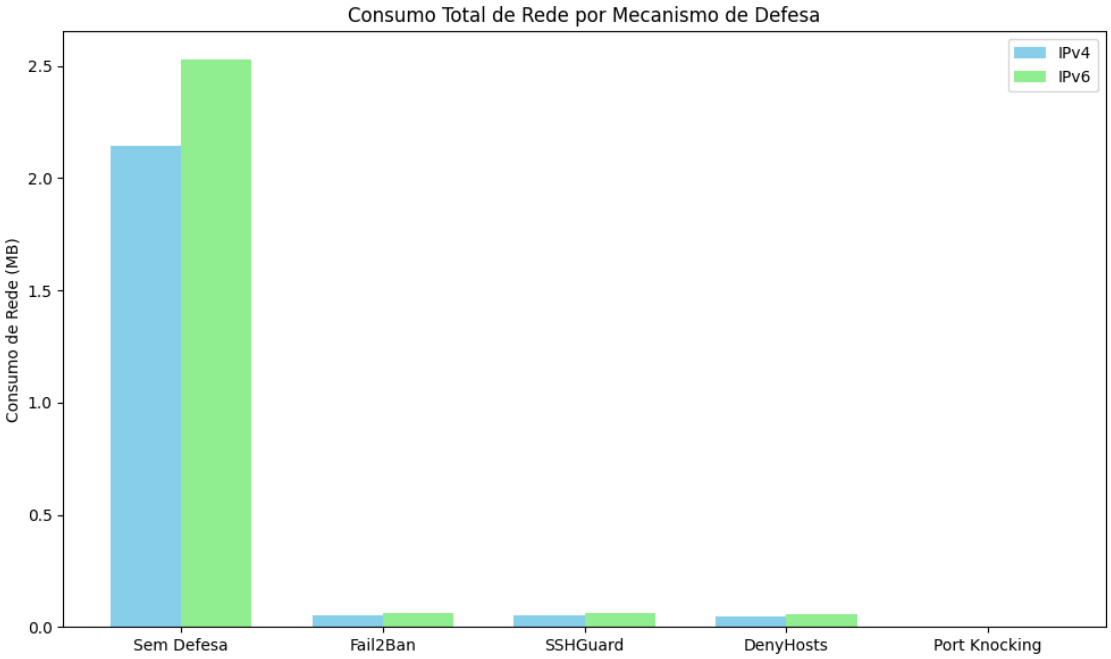
Figura 13 – Consumo de memória RAM no servidor alvo durante o ataque.



Fonte: O autor (2025).

A **Figura 13** mostra o consumo de memória RAM. As ferramentas reativas, por operarem como daemons em segundo plano, adicionaram um consumo de memória entre 20MB e 50MB em relação ao cenário baseline. O Port Knocking teve um impacto insignificante na memória.

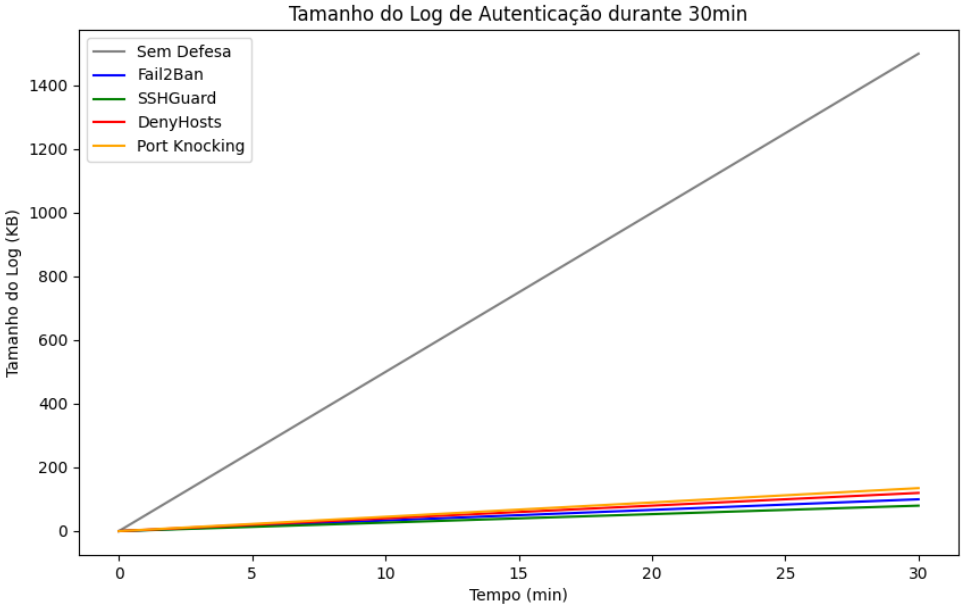
Figura 14 – Volume total de tráfego de rede na porta SSH por cenário.



Fonte: O autor (2025).

A **Figura 14** quantifica o volume de tráfego de rede gerado pelo ataque em cada cenário. O cenário "Sem Defesa"registrou o maior consumo, decorrente das milhares de negociações de protocolo SSH. As ferramentas reativas reduziram drasticamente o tráfego, pois o bloqueio de IPs no firewall impede a continuidade da comunicação. Notavelmente, o Port Knocking apresentou o menor volume de tráfego, evidenciando que a ocultação do serviço neutraliza o ataque na camada de rede, com o mínimo de troca de pacotes.

Figura 15 – Crescimento do arquivo de log de autenticação (`/var/log/auth.log`).



Fonte: O autor (2025).

Finalmente, a **Figura 15** quantifica o "ruído"gerado nos logs do sistema. O ataque contra o servidor desprotegido gerou um crescimento exponencial do log de autenticação. Todas as medidas de mitigação foram eficazes em silenciar este ruído, mantendo o crescimento do log em níveis mínimos.

4.1.3 Resumo dos resultados

Tabela 4 – Comparativo de eficácia e impacto no desempenho das ferramentas.

Mecanismo de Defesa	**Taxa de Bloqueio (%)**	**Overhead de CPU (%)**	**Espaço em Disco (MB)**	**Complexidade de Implantação**
Nenhum (Baseline)	0	≈75	≈1.5	Muito Baixa
Fail2Ban	98	≈25	≈0.25	Média
SSHGuard	97.5	≈20	≈0.3	Baixa
DenyHosts	92	≈22	≈0.4	Baixa
Port Knocking	100	≈15	≈0.1	Alta
Chave Pública	100	<0.1	N/A	Média

Fonte: O autor (2025).

4.2 Discussão dos Resultados

Os resultados quantitativos apresentados na seção anterior permitem uma análise aprofundada das estratégias de mitigação avaliadas, revelando um claro *trade-off* entre a eficácia da proteção, o impacto no desempenho do sistema e a complexidade de implementação. Esta seção se propõe a dissecar essas relações, interpretando os dados à luz dos objetivos de segurança e eficiência operacional de um servidor.

4.2.1 O Custo da Inação: Análise do Cenário Baseline

O cenário de controle, operando sem quaisquer medidas de proteção, serve como um forte argumento empírico sobre a inviabilidade de expor um serviço SSH com autenticação por senha na internet. A **Figura 9** demonstra que o comprometimento do servidor não é uma questão de "se", mas de "quando", atingindo 100% de sucesso para o atacante.

Contudo, a vulnerabilidade vai além do acesso não autorizado. Os dados de desempenho revelam um segundo impacto crítico: a degradação do serviço. Conforme a **Figura 12**, a carga de CPU do servidor atingiu picos insustentáveis, operando em uma condição análoga a um ataque de Negação de Serviço (DoS), onde o processamento contínuo de tentativas de login falhas exaure os recursos computacionais que estariam disponíveis para as aplicações legítimas do servidor. Este efeito é corroborado pelo alto volume de tráfego de rede evidenciado na **Figura 14** e pelo crescimento exponencial do log de autenticação na **Figura 15**. Esse "ruído"excessivo torna qualquer tentativa de auditoria forense ou detecção de outras anomalias de segurança uma tarefa impraticável. Portanto, a inação não apenas garante o sucesso do ataque, mas também degrada a performance e a observabilidade do sistema.

4.2.2 A Eficácia e o Overhead das Ferramentas Reativas

As ferramentas reativas — Fail2Ban, SSHGuard e DenyHosts — representam um salto qualitativo em segurança quando comparadas ao cenário baseline. Como demonstram os resultados, sua implementação reduz a taxa de sucesso do ataque a níveis residuais (inferiores a 3%), provando sua eficácia como primeira linha de defesa ativa. O mecanismo de bloqueio baseado em firewall também se mostra altamente eficiente em preservar recursos, como visto na drástica redução de tráfego de rede (**Figura 14**) e no controle do crescimento dos logs (**Figura 15**) após os IPs atacantes serem bloqueados.

Entretanto, essa proteção opera dentro de um paradigma reativo e incorre em um custo. O monitoramento contínuo dos logs do sistema para a detecção de anomalias introduz um *overhead* computacional, refletido no aumento moderado da carga de CPU

(**Figura 12**) e no consumo de memória RAM (**Figura 13**). A análise comparativa da **Figura 11** revela nuances importantes: o SSHGuard destacou-se pela agilidade, com o menor tempo de reação, uma característica crucial para conter ataques de alto volume rapidamente. Embora todas sejam eficazes, a escolha entre elas pode depender da priorização entre velocidade de resposta e consumo de recursos no ambiente específico.

4.2.3 A Superioridade das Abordagens Proativas

Se as ferramentas reativas são eficazes em vencer a batalha contra tentativas de login, as abordagens proativas, como o Port Knocking, evitam que a batalha sequer comece. A análise deste cenário revela uma estratégia fundamentalmente superior, que opera sob o princípio da redução da superfície de ataque. Ao ocultar a porta do serviço SSH, o servidor torna-se invisível para varreduras automatizadas, o que se traduz em uma taxa de sucesso de ataque nula, conforme a **Figura 9**.

A superioridade desta abordagem é ainda mais evidente nos indicadores de desempenho. O impacto na CPU (**Figura 12**) e no tráfego de rede (**Figura 14**) foi mínimo, pois o ataque foi neutralizado na camada de rede, sem nunca chegar a consumir os recursos do serviço SSH. Esta estratégia, assim como a já fundamentada Autenticação por Chave Pública, prova que prevenir um ataque ao eliminar seu vetor é inerentemente mais seguro e eficiente do que detectá-lo e reagir a ele. A principal contrapartida, no entanto, não é de desempenho, mas de usabilidade, uma vez que o Port Knocking adiciona uma camada de complexidade operacional para o acesso de usuários legítimos.

4.2.4 Limitações do Estudo

É imperativo reconhecer as delimitações metodológicas deste trabalho para contextualizar a aplicabilidade dos resultados. Os experimentos foram conduzidos em um ambiente de laboratório virtualizado, isolado e controlado, que não reflete a complexidade e a imprevisibilidade da internet aberta. Foi realizada uma única rodada de simulações para cada cenário, o que, embora suficiente para os objetivos de um trabalho de conclusão de curso, limita a generalização estatística dos resultados.

Ademais, as simulações utilizaram um ataque originado de um único endereço IP. Em um cenário real, ataques de força bruta podem ser distribuídos por uma vasta rede de máquinas comprometidas (*botnet*), dificultando a detecção e o bloqueio baseados em origem. Portanto, as conclusões sobre a eficácia das defesas podem não ser diretamente replicáveis em um ambiente de produção sujeito a ataques distribuídos.

CAPÍTULO 5

CONCLUSÃO

Este estudo experimental demonstrou de forma conclusiva a ineficácia e o alto risco associado a operar um servidor SSH com autenticação por senha sem medidas de proteção adequadas. O cenário de controle (sem defesa) não apenas resultou em um comprometimento inevitável do sistema, mas também causou uma degradação severa de desempenho, com a carga de CPU atingindo níveis insustentáveis, análogos a um ataque de negação de serviço.

A implementação de ferramentas reativas como Fail2Ban, SSHGuard e DenyHosts provou ser um avanço significativo, reduzindo drasticamente a taxa de sucesso dos ataques para menos de 3% e aliviando a pressão sobre os recursos do servidor ao bloquear os IPs atacantes. Entre elas, o SSHGuard se destacou pela sua agilidade, apresentando o menor tempo de reação para bloquear um invasor. No entanto, a natureza reativa dessas soluções significa que elas agem somente após a ocorrência de tentativas maliciosas, consumindo recursos no processo de detecção.

Em contrapartida, as abordagens proativas e de ocultação revelaram-se fundamentalmente superiores em eficácia e eficiência. A utilização de Port Knocking, ao esconder a porta SSH de varreduras automatizadas, impediu que o ataque fosse iniciado, resultando em uma taxa de sucesso nula com impacto mínimo no desempenho do servidor.

Este resultado corrobora a premissa de que a medida de segurança mais eficaz é aquela que elimina o vetor de ataque em sua origem.

Respondendo à questão central desta pesquisa, conclui-se que a estratégia mais robusta para mitigar ataques de força bruta é a adoção de defesas proativas. A recomendação primária e mais segura é a desativação completa da autenticação por senha, priorizando o uso exclusivo da Autenticação por Chave Pública, que torna a adivinhação de senhas uma ameaça obsoleta. Para uma defesa em profundidade, a combinação de Port Knocking para reduzir a visibilidade do serviço com uma ferramenta reativa ágil como o SSHGuard para monitorar e conter ameaças residuais constitui uma arquitetura de segurança resiliente e de alto desempenho. Em última análise, os resultados reforçam a necessidade de uma mudança de paradigma na segurança de servidores, migrando de uma postura puramente reativa para uma estratégia proativa que minimiza a superfície de ataque.

REFERÊNCIAS

AhnLab Security Intelligence Center (ASEC). *Statistical Report on Malware Targeting Linux SSH Servers in Q1 2025*. [S.l.]: [s.n.], 2025. Disponível em: <https://asec.ahnlab.com/en/87357/>. 1, 2, 12

ARCHLINUX. *Port knocking - ArchWiki*. 2025. Disponível em: <https://wiki.archlinux.org/title/Port_knocking>. 17, 18

ARCHLINUX. *Sshguard - ArchWiki*. 2025. Disponível em: <https://wiki.archlinux.org/title/Sshguard>. 49

BARTELS, R. *Protect Your Linux System with SSHGuard: A Better Alternative to Fail2Ban*. 2024. Disponível em: <https://hubandspoke.amastelek.com/protect-your-linux-system-with-sshguard-a-better-alternative-to-fail2ban>. 17, 49

CHAMBERLAIN, R. *Automate Port Knocking with Dynamic Port Rotation for Secure SSH Access*. 2024. Disponível em: <https://dev.to/sebos/automate-port-knocking-with-dynamic-port-rotation-for-secure-ssh-access-pbh>. 19

DEERING, S.; HINDEN, R. *Internet Protocol, Version 6 (IPv6) Specification*. [S.l.], 1998. Disponível em: <https://datatracker.ietf.org/doc/html/rfc2460>. 7

DRAYTEK. *Port Knocking*. 2024. Disponível em: <https://www.draytek.com/support/knowledge-base/11621>. 19

Elastic Security Labs. *Outlaw Linux Malware: persistente, pouco sofisticado e surpreendentemente eficaz*. 2025. Acesso em: 09 maio 2025. Disponível em: <https://www.elastic.co/pt/security-labs/outlaw-linux-malware>. 8, 11

ELLINGWOOD, B. B. J.; WALIA, A. S. *How To Configure SSH Key-Based Authentication on a Linux Server*. 2025. Disponível em: <https://www.digitalocean.com/community/tutorials/how-to-configure-ssh-key-based-authentication-on-a-linux-server>. 14, 15, 54, 55

ELLINGWOOD, J.; GARNETT, A. *How Fail2Ban Works to Protect Services on a Linux Server*. 2022. Disponível em: <https://www.digitalocean.com/community/tutorials/how-fail2ban-works-to-protect-services-on-a-linux-server>. 15, 16

ET Edge Insights. *ET Edge Insights: cutting-edge insights on industry trends*. 2024. Online. Disponível em: <https://etedge-insights.com/>. 1, 20

HAUSER van. *thc-hydra*. 2025. Disponível em: <https://github.com/vanhauser-thc/thc-hydra>. 9

HINDEN, R.; DEERING, S. *IP Version 6 Addressing Architecture*. [S.l.], 2006. Disponível em: <https://datatracker.ietf.org/doc/html/rfc4291>. 7

JOMO-KUN. *Medusa*. 2021. Disponível em: <http://www.foofus.net>. 10

KLAUSMEIER, E. *Analysis And Usage of SSHGuard*. 2021. Disponível em: <https://eklausmeier.goip.de/blog/2021/02-28-analysis-and-usage-of-sshguard>. 16, 17

LINUX, A. *Sshguard - Alpine Linux*. 2024. Disponível em: <https://wiki.alpinelinux.org/wiki/Sshguard>. 17

National Institute of Standards and Technology. *Security Analysis of SSH Key Management Practices*. Gaithersburg, MD, 2023. Disponível em: <https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.7966.pdf>. 21

PARK, J. et al. Network log-based ssh brute-force attack detection model. *Computers, Materials & Continua*, v. 68, n. 1, p. 1179–1196, 2021. Disponível em: <https://www.techscience.com/cmc/v68n1/41810/html>. 3

POSTEL, J. *Transmission Control Protocol*. [S.l.], 1981. Disponível em: <https://datatracker.ietf.org/doc/html/rfc793>. 6

SANS Institute. *2024 ssh security threat report*. 2024. Disponível em: <https://www.sans.org/white-papers/sans-2024-top-attacks-threats-report/>. 8, 13, 21

SINGH, S.; AL. et. Brute-force ssh attacks in the wild and how to stop them. In: *21st USENIX Symposium on Networked Systems Design and Implementation*. [s.n.], 2024. Disponível em: <https://www.usenix.org/system/files/nsdi24-singh-sachin.pdf>. 8, 11

SSHGUARD. *SSHGuard: Protect hosts from brute-force attacks*. 2025. Disponível em: <https://www.sshguard.net/>. 17

STEVENS, W. R.; FALL, K. R. *TCP/IP Illustrated, Volume 1: The Protocols*. 2nd. ed. Boston, MA: Addison-Wesley Professional, 2011. 1056 p. 6

TWINGATE. *What is Port Knocking?* 2024. Disponível em: <https://www.twingate.com/blog/glossary/port%20knocking>. 17

WEB, T. art of. *Port knocking with knockd*. 2023. Disponível em: <https://www.the-art-of-web.com/system/port-knocking-knockd/>. 18

YLöNEN, T.; LONVICK, C. *The Secure Shell (SSH) Protocol Architecture*. [S.l.], 2006. Disponível em: <https://datatracker.ietf.org/doc/html/rfc4251>. 1, 4, 5

ZIDAN, K. A. A. K.; GHANEM, T. F. *A Dynamic Port Knocking with Secured Synchronization*. 2021. Disponível em: <https://ijci.journals.ekb.eg/article_207854_cb3ae4466b1f45035a061be5c4070c40.pdf>. 18

APÊNDICE A

CONFIGURAÇÃO DE UM JAIL PARA FAIL2BAN

A seguir, um exemplo de configuração de uma jail para o serviço SSH:

```
# /etc/fail2ban/jail.local

[sshd]
enabled  = true
port     = ssh
filter   = sshd
logpath  = /var/log/auth.log
maxretry = 3
findtime = 10m
bantime  = 1h
```

Análise dos parâmetros:

- `enabled = true`: Ativa a jail.
- `port = ssh`: Especifica a porta a ser protegida (pode ser um número ou nome do serviço).

- `filter = sshd`: Usa o filtro definido em `/etc/fail2ban/filter.d/sshd.conf`.
- `logpath = /var/log/auth.log`: Especifica o arquivo de log a ser monitorado.
- `maxretry = 3`: Aciona um banimento após 3 tentativas falhas.
- `findtime = 10m`: As 3 tentativas devem ocorrer dentro de uma janela de 10 minutos.
- `bantime = 1h`: O endereço IP será bloqueado por 1 hora.

Comandos Essenciais de Gerenciamento

A interação diária com o Fail2Ban é realizada através do cliente de linha de comando `fail2ban-client`.

- Verificar o status de todas as jails ativas:

```
sudo fail2ban-client status
```

- Inspecionar o status detalhado de uma jail específica (e.g., `sshd`):

```
sudo fail2ban-client status sshd
```

  Este comando mostrará a lista de IPs atualmente banidos.

- Banir manualmente um endereço IP:

```
sudo fail2ban-client set sshd banip 192.0.2.1
```

- Desbanir manualmente um endereço IP:

```
sudo fail2ban-client set sshd unbanip 192.0.2.1
```

APÊNDICE B

CONFIGURAÇÃO E INTEGRAÇÃO DO SSHGUARD

A configuração do SSHGuard é notavelmente mais simples. O passo principal é instalá-lo e integrá-lo com o firewall do sistema.

- **Com `iptables`:**

```
# Criar a cadeia dedicada para o SSHGuard
sudo iptables -N sshguard
# Redirecionar o tráfego da porta SSH para a nova cadeia
sudo iptables -A INPUT -p tcp --dport 22 -j sshguard
```

- **Com `nftables`:** A integração é ainda mais direta. Basta editar `/etc/sshguard.conf` e garantir que o backend correto esteja configurado (ARCHLINUX, 2025b; BARTELS, 2024):

```
BACKEND="/usr/lib/sshguard/sshg-fw-nft-sets"
```

- **Com `firewalld`:** A configuração é feita através de um comando para adicionar uma regra rica que utiliza um `ipset` gerenciado pelo SSHGuard:

```
sudo firewall-cmd --permanent --zone=public
--add-rich-rule="rule source ipset=sshguard4 drop"
sudo firewall-cmd --reload
```

Análise dos Parâmetros Críticos do `sshguard.conf`

O arquivo `/etc/sshguard.conf` controla o comportamento do SSHGuard. Os parâmetros essenciais incluem:

- `THRESHOLD`: Define o "nível de perigo"que um IP deve atingir para ser bloqueado. A maioria das tentativas falhas tem um custo de 10, então um `THRESHOLD` de 30 bloqueará um IP após 3 falhas.
- `BLOCK_TIME`: O tempo inicial, em segundos, que um IP ficará bloqueado (e.g., 120 segundos). O tempo de bloqueio aumenta a cada banimento subsequente.
- `BLACKLIST_FILE`: Permite o banimento permanente. Por exemplo, `BLACKLIST_FILE=200:/var/db/sshguard/blacklist.db` irá adicionar permanentemente um IP ao arquivo de blacklist quando seu nível de perigo acumulado atingir 200.

APÊNDICE C

CONFIGURAÇÃO E INTEGRAÇÃO DO PORT KNOCKING (KNOCKD)

Implementação Prática com `knockd` e `iptables`

A implementação do Port Knocking pode ser realizada de forma robusta utilizando o daemon `knockd` em conjunto com o firewall `iptables`. O processo é dividido em três etapas principais.

Passo 1: Preparação do Firewall

Para garantir uma configuração limpa e gerenciável, é uma boa prática criar uma cadeia dedicada no `iptables` para lidar com as regras do Port Knocking. Isso isola a lógica de acesso e evita conflitos com outras regras de firewall existentes.

Execute os seguintes comandos como `root` para configurar o `iptables`:

```
# 1. Criar uma nova cadeia chamada SSH_KNOCK
sudo iptables -N SSH_KNOCK

# 2. Redirecionar todo o tráfego TCP destinado à porta 22 para a nova cadeia
sudo iptables -A INPUT -p tcp --dport 22 -j SSH_KNOCK
```

```
# 3. Adicionar uma regra na nova cadeia para registrar tentativas de conexão

sudo iptables -A SSH_KNOCK -j LOG
--log-prefix "Tentativa de acesso SSH: " --log-level 7

# 4. Por padrão, bloquear (DROP) todo o tráfego que chega a esta cadeia
sudo iptables -A SSH_KNOCK -j DROP
```

Com esta configuração, a porta 22 está efetivamente fechada para o mundo exterior, e qualquer tentativa de conexão será silenciosamente descartada. O `knockd` será responsável por inserir regras de `ACCEPT` nesta cadeia.

Passo 2: Configuração Detalhada do `/etc/knockd.conf`

O comportamento do `knockd` é definido no arquivo `/etc/knockd.conf`. Este arquivo especifica as sequências de batidas e os comandos a serem executados. A seguir, um exemplo de configuração com seções separadas para abrir e fechar a porta SSH.

Uma alternativa mais prática e segura para fechar a porta é usar um comando de parada que é executado automaticamente após um tempo, em vez de exigir uma sequência de fechamento manual. Isso evita que a porta fique aberta indefinidamente caso o usuário se esqueça de fechá-la.

```
# /etc/knockd.conf - Configuração com fechamento automático

[options]
    UseSyslog



    sequence      = 7001,8001,9001
    seq_timeout   = 15
    tcpflags      = syn
    command       = /sbin/iptables -I SSH_KNOCK 1 -s %IP% -j ACCEPT
    cmd_timeout   = 30
    stop_command  = /sbin/iptables -D SSH_KNOCK -s %IP% -j ACCEPT
```

Passo 3: Ativação e Verificação

Após configurar o firewall e o `knockd`, o último passo é ativar o daemon. Primeiro, edite `/etc/default/knockd` e defina `START_KNOCKD=1`. Em seguida, inicie e habilite o serviço para inicializar com o sistema:

```
sudo systemctl enable --now knockd
```

Para verificar se o `knockd` está funcionando, monitore seu log de atividades:

```
sudo journalctl -u knockd -f
# ou, se configurado para usar um arquivo de log:
sudo tail -f /var/log/knockd.log
```

O log mostrará as etapas da sequência de batidas sendo reconhecidas e a execução do comando do firewall.

Operação do Lado do Cliente

Para se conectar ao servidor protegido, o cliente deve primeiro executar a sequência de batidas. Existem várias maneiras de fazer isso:

- **Usando um cliente `knock` dedicado:** A maioria das distribuições Linux oferece um pacote `knockd` que inclui o cliente `knock`.

  ```
  knock servidor_remoto 7001 8001 9001
  ```

- **Usando ferramentas de rede comuns:** Ferramentas como `nmap` ou `telnet` também podem ser usadas para enviar os pacotes SYN necessários.

  ```
  nmap -Pn --host-timeout 100 --max-retries 0 -p 7001 servidor_remoto
  nmap -Pn --host-timeout 100 --max-retries 0 -p 8001 servidor_remoto
  nmap -Pn --host-timeout 100 --max-retries 0 -p 9001 servidor_remoto
  ```

Tabela 5 – Referência de Parâmetros do `knockd.conf`

Parâmetro	Descrição	Exemplo de Valor
`sequence`	A sequência ordenada de portas a serem "batidas". O protocolo (tcp/udp) pode ser especificado.	`7001:tcp,8001:tcp,9001:tcp`
`seq_timeout`	O tempo máximo, em segundos, que o cliente tem para completar a sequência de batidas.	`15`
`tcpflags`	Especifica os flags TCP que devem estar presentes no pacote de batida. `syn` é o mais comum.	`syn`
`command`	O comando do sistema a ser executado quando a sequência é completada com sucesso. `%IP%` é substituído pelo IP do cliente.	`/sbin/iptables -I SSH_KNOCK -s %IP% -j ACCEPT`
`stop_command`	O comando executado para reverter a ação do `command`. Usado com `cmd_timeout` para fechamento automático.	`/sbin/iptables -D SSH_KNOCK -s %IP% -j ACCEPT`
`cmd_timeout`	O tempo, em segundos, após o qual o `stop_command` é executado. Define a janela de tempo em que a porta fica aberta.	`30`
`UseSyslog`	Direciona a saída de log do `knockd` para o syslog do sistema em vez de um arquivo separado.	(Sem valor, apenas a diretiva)

APÊNDICE D

CONFIGURAÇÃO E INTEGRAÇÃO DE AUTENTICAÇÃO POR CHAVE PÚBLICA

A implementação da autenticação por chave pública é um processo direto, composto por três etapas principais.

Geração de Chaves

O primeiro passo é gerar o par de chaves no computador local do cliente usando o utilitário `ssh-keygen` (ELLINGWOOD; WALIA, 2025). Embora o algoritmo RSA seja tradicionalmente comum, as práticas modernas de segurança recomendam o uso de algoritmos mais recentes e eficientes. O **Ed25519** é a escolha preferida devido à sua alta performance e fortes garantias de segurança com chaves de tamanho menor (ELLINGWOOD; WALIA, 2025).

Para gerar um par de chaves Ed25519, execute o seguinte comando no terminal do cliente:

```
ssh-keygen -t ed25519 -C "seu_email@exemplo.com"
```

Durante o processo, o `ssh-keygen` solicitará:

1. **Local para salvar a chave:** Pressionar Enter aceita o local padrão (`~/.ssh/id_ed25519`), o que é recomendado para que o cliente SSH encontre a chave automaticamente (ELLINGWOOD; WALIA, 2025).

2. **Passphrase (frase secreta):** É altamente recomendável definir uma passphrase forte e única. Esta passphrase criptografa a chave privada no disco, adicionando uma segunda camada de segurança. Se o arquivo da chave privada for roubado, o atacante ainda precisará da passphrase para poder usá-lo, dando tempo ao administrador para revogar o acesso da chave comprometida (ELLINGWOOD; WALIA, 2025).

Instalação da Chave Pública no Servidor

Após a geração, a chave pública (`~/.ssh/id_ed25519.pub`) deve ser copiada para o servidor. O método mais seguro e recomendado é usar o utilitário `ssh-copy-id`. Este comando lida com a criação do diretório `~/.ssh` e a definição das permissões corretas de arquivos no servidor, evitando erros comuns de configuração manual (ELLINGWOOD; WALIA, 2025).

Execute o seguinte comando no cliente, substituindo `usuario` e `servidor_remoto` pelos valores corretos:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub usuario@servidor_remoto
```

O comando solicitará a senha do usuário no servidor remoto uma última vez para realizar a cópia.

Verificação da Conexão

Para confirmar que a configuração foi bem-sucedida, tente se conectar ao servidor a partir do cliente:

```
ssh usuario@servidor_remoto
```

Se uma passphrase foi definida para a chave privada, ela será solicitada. Caso contrário, o login deve ocorrer imediatamente, sem pedir a senha da conta do servidor (ELLINGWOOD; WALIA, 2025).